# Comprehensive Speaker Notes
# AgentClinic v2.1 — M.Tech Thesis Viva

Dikshant Sharma — IIT Kharagpur

April 2026

## Contents


1 Slide 1: Title Slide 3

2 Slide 2: Introduction — The Problem with Static Benchmarks 4

3 Slide 3: Introduction — The Research Goal 5

4 Slide 4: Literature Review — Static vs. Dynamic Evaluation 6

5 Slide 5: The AgentClinic-MedQA Dataset 8

6 Slide 6: Phase 1 — Initial Framework Architecture 9

7 Slide 7: Phase 1 — Baseline Results 11

8 Slide 8: The Discovery of Lexical Leakage 12

9 Slide 9: Phase 1 Limitations — Motivation for Redesign 14

10 Slide 10: AgentClinic v2 — Enhanced Architecture (LangGraph DCG) 15

11 Slide 11: Eliminating Leakage — Heterogeneous Multi-Engine Setup 18

12 Slide 12: The Metacognitive Reflection Node 20

13 Slide 13: AgentClinic v2 — System Flowchart 22

14 Slide 14: Process-Oriented Clinical Metrics 23

15 Slide 15: Structured Cognitive Bias Injections 25

16 Slide 16: Dynamic LoRA Adapter Routing 27

17 Slide 17: E1–E3 — Baseline and Foundational Outcomes 29

18 Slide 18: E6 — Dynamic LoRA Results 31

19 Slide 19: Dynamic LoRA — Architecture Deep-Dive 33

**20 Slide 20: The Safety Threat of Cognitive Biases** **34**

**21 Slide 21: Discussion — Specialisation vs. Alignment** **36**

**22 Slide 22: Conclusion** **38**

**23 Slide 23: Future Work** **40**

# 1 Slide 1: Title Slide

## Timing

*Target: 0.5 minutes — Cumulative: 0.5 minutes*

## Spoken Script

Good morning, respected committee members. My name is Dikshant Sharma, and I am an M.Tech student in Artificial Intelligence at the Department of Computer Science and Engineering, IIT Kharagpur. [PAUSE]

The title of my thesis is "Design and Evaluation of Multi-Agent LLM Systems for Interactive Clinical Diagnosis." [SLOW DOWN] Over the next thirty minutes, I will walk you through a two-phase research programme that began with a fundamental question: can we trust the clinical reasoning of open-source language models—not just their ability to answer medical trivia, but their capacity to conduct a real, interactive patient consultation? [PAUSE]

This work was carried out under the guidance of Professor Jiaul Hoque Paik. Let me begin with a brief roadmap of the presentation.

## Technical Deep-Dive (Internal Reference)

No deep technical content needed for the title slide. Key identifiers to remember if asked: M.Tech in AI programme, academic year 2025–2026, thesis supervised within the CSE department at IIT Kharagpur under Prof. Jiaul Hoque Paik.

## Transition to Next Slide

*Let me begin with the fundamental problem that this thesis addresses.*

## Anticipated Examiner Questions

**Q1:** What motivated you to choose this particular topic?
**A1:** I was struck by the disconnect between the headline numbers—GPT-4 scoring 90% on USMLE—and the complete absence of evidence that these models can actually conduct a patient interview. Static benchmarks test recall, not reasoning. I wanted to build a rigorous framework to test the latter.

**Q2:** Is this an extension of an existing framework or entirely your own?
**A2:** The initial codebase, AgentClinic, is an open-source multi-agent framework from prior work. My contribution was identifying its architectural flaws—most notably Lexical Leakage—and rebuilding the entire system as a LangGraph state machine with novel metacognitive reflection, process metrics, and dynamic LoRA routing. The enhanced framework is my own design.

# 2 Slide 2: Introduction — The Problem with Static Benchmarks

## Timing

*Target: 2 minutes — Cumulative: 2.5 minutes*

## Spoken Script

Let me start with the core tension that motivates this entire thesis. [POINT TO SLIDE] You see four bullet points on this slide, but the story beneath them is critical. [PAUSE]

Traditional clinical AI evaluation uses multiple-choice benchmarks: MedQA, USMLE, PubMedQA. In these tests, the model receives everything upfront—the full patient history, every lab result, every imaging finding. It simply selects an answer from multiple choices. GPT-4 scores above 90% on USMLE under these conditions. That sounds impressive. [PAUSE]

But real clinical reasoning is fundamentally different. A physician sits with a patient who says "I have a headache." The physician does not know the full history yet. They must ask targeted questions, form hypotheses, order specific tests, and iteratively refine their differential diagnosis—all under genuine uncertainty. [PAUSE]

[SLOW DOWN] The gap between declarative factual recall and procedural clinical reasoning is the core problem. Static benchmarks provide a fully structured vignette upfront. Real medicine requires iterative hypothesis generation and sequential test ordering in the face of uncertainty. A model scoring highly on MedQA is not necessarily capable of conducting a safe, multi-turn clinical consultation. My thesis is built entirely on addressing this gap.

## Technical Deep-Dive (Internal Reference)

**Benchmark specifics:** MedQA contains 12,723 questions from Chinese medical licensing exams; USMLE contains questions from the United States Medical Licensing Examination. PubMedQA focuses on biomedical research abstracts. All share the same fundamental limitation: single-turn, closed-book (or open-book) QA format.

**GPT-4 USMLE performance:** Nori et al. (2023) reported GPT-4 achieving 86.7% on USMLE Step 1, 2, and 3 combined. However, these exams are designed as closed-form MCQs where all relevant information is bundled into a single vignette. The model never needs to *acquire* information—it only needs to *process* pre-given data.

**Why this matters for patient safety:** In a real clinical encounter, a model that cannot conduct sequential history-taking may miss critical symptoms, order unnecessary invasive tests, or anchor on the first plausible diagnosis without exploring alternatives.

## Transition to Next Slide

*This gap is not merely academic—it leads directly to the research questions I set out to answer.*

### Anticipated Examiner Questions

**Q1:** Are there no interactive medical AI benchmarks at all?
**A1:** There are emerging efforts. Google's AMIE system demonstrated multi-turn consultation, but it is entirely closed-source and non-reproducible. Agent Hospital by Li et al. uses multi-agent simulation, but its homogeneous architecture introduces the Lexical Leakage vulnerability I will describe shortly. My work builds on AgentClinic specifically to address these gaps with an open, reproducible, and architecturally sound framework.

**Q2:** Isn't USMLE performance still a useful signal?
**A2:** Absolutely—it validates medical knowledge retrieval. But it is a necessary, not sufficient, condition for clinical competence. A model that scores 90% on USMLE but cannot manage a ten-turn patient interview is not clinically deployable. My framework specifically tests the interactive dimension.

## 3 Slide 3: Introduction — The Research Goal

### Timing

*Target: 2 minutes — Cumulative: 4.5 minutes*

### Spoken Script

The central objective of this thesis is to design, implement, and rigorously evaluate an interactive multi-agent simulation framework that tests open-source LLMs not just on *what* they diagnose, but on *how* they reason. [PAUSE]

These four core research questions drove every architectural and experimental decision. [PAUSE]

RQ1 asks: can open-source models in the 7 to 8 billion parameter range actually conduct safe multi-turn clinical interviews using only local GPU inference? This is a feasibility question. [PAUSE]

RQ2 addresses the vulnerability I discovered: how do we prevent models from "cheating" by implicitly accessing ground-truth data in a simulated environment? If homogeneous architectures leak information, then previous benchmarks are unreliable. [PAUSE]

RQ3 probes safety: [SLOW DOWN] how do cognitive biases affect an LLM's diagnostic reasoning trajectory? Because if we deploy these models in hospitals, the same recency and confirmation biases that affect human doctors will also affect them. [PAUSE]

And RQ4 is the architecture question: can architectural innovations—deterministic state machines, metacognitive reflection, and adapter routing—improve LLM clinical reasoning beyond what model scaling alone achieves? This is the central thesis of the work, and experiments E1 through E6 provide a definitive answer.

### Technical Deep-Dive (Internal Reference)

**RQ1 → E1:** Mistral-7B-Instruct-v0.2 running on a single NVIDIA H100 NVL under NF4 quantization. Proves feasibility of local inference for multi-turn clinical simulation.

**RQ2 → E1 vs Phase 1:** Phase 1 accuracy was 43.9% with homogeneous weights (Lexical Leakage present). E1 achieves 45.8% with heterogeneous weights (Lexical Leakage removed) *plus* the reflection node. The net increase despite removing an artificial advantage proves that the reflection node compensates.

**RQ3 → E4, E5:** Recency bias uses $k = 5$ recent case summaries injected into the Doctor's system prompt. Confirmation bias primes the Doctor with "You are initially confident that the patient has cancer." Both exploit the transformer's self-attention mechanism.

**RQ4 → E1–E6:** The entire experimental matrix answers this question. The LangGraph DCG (deterministic state machine), the metacognitive reflection node, and Dynamic LoRA adapter routing (E6) collectively demonstrate that architectural innovations yield a 43.9% → 55.1% improvement trajectory—far beyond what model scaling alone achieves.

Each question maps directly to specific experiments: RQ1 to E1, RQ2 to E1 versus Phase 1, RQ3 to experiments E4 and E5, and RQ4 to the full E1–E6 progression.

### Transition to Next Slide

*These questions emerged from studying the existing landscape. Let me show you what the literature reveals about the current state of clinical AI evaluation.*

### Anticipated Examiner Questions

**Q1:** Were these research questions formulated upfront or did they emerge during the work?
**A1:** RQ1 and RQ4 were formulated at the outset. RQ2 emerged during Phase 1 when I noticed unexpectedly high accuracy and traced it to Lexical Leakage. RQ3 was formulated during the Phase 2 redesign as a direct response to the memory constraints of running multiple models. So the research questions evolved with the discoveries.

**Q2:** Why focus on 7B–8B models instead of larger ones?
**A2:** Two reasons. First, practical accessibility: a 70B model requires multiple GPUs and is impractical for most research labs. Second, scientific control: by keeping parameter counts roughly constant across experiments, performance deltas are attributable to training paradigm and architecture, not raw model size. The 7B range is the sweet spot where emergent reasoning abilities appear while remaining manageable with NF4 quantization on a single GPU.

## 4 Slide 4: Literature Review — Static vs. Dynamic Evaluation

### Timing

*Target: 2 minutes — Cumulative: 6.5 minutes*

### Spoken Script

Let me place this work in context with three observations from the literature. [PAUSE]

First: proprietary systems lead, open-source lags. Google's AMIE demonstrates

expert-level multi-turn reasoning, but it is entirely closed-source. No one outside Google can reproduce, extend, or audit its reasoning. Agent Hospital uses the same model for all agents, which introduces the vulnerability I discovered. MedAgents by Tang et al. is static; it does not support multi-turn simulation at all. [PAUSE]

Second: there is no established methodology for quantifying the *stability* or *rationality* of an LLM's reasoning process across a multi-turn encounter. Outcome accuracy remains the sole metric. [PAUSE]

[SLOW DOWN] But here is the truly dangerous part. A language model can appear *confidently consistent*—giving stable, authoritative-sounding diagnoses—while being *systematically wrong*. Binary accuracy alone cannot detect this. If the model orders an MRI before checking blood pressure, that is an unsafe reasoning trajectory that a simple "correct or incorrect" metric completely misses. [PAUSE]

[MAKE EYE CONTACT] This is the evaluative gap my thesis fills: not just *what* the model concludes, but *how* it got there.

## Technical Deep-Dive (Internal Reference)

**AMIE (Articulate Medical Intelligence Explorer):** Published by Tu et al. (2024) at Google. Uses a proprietary fine-tuned PaLM 2 model. Achieves strong results in simulated consultations but cannot be reproduced or extended by external researchers. No weight access, no API access for research.

**Agent Hospital (Li et al., 2024):** Uses the same model instance for all agent roles (Doctor, Patient, etc.). This creates a shared embedding manifold—the mathematical space in which the model encodes text. My Lexical Leakage finding shows that when both Doctor and Patient project text into identical latent spaces, the Doctor can implicitly "recognise" the statistical fingerprint of the Patient's generated text, gaining unfair access to ground-truth information.

**MedAgents (Tang et al., 2023):** Uses a role-playing framework where multiple LLM instances take on specialist roles (cardiologist, neurologist, etc.) to collaboratively answer a single static medical question. No multi-turn patient interaction.

**The "confidently wrong" pattern:** This is precisely what E5 (Confirmation Bias) demonstrates later—DS = 0.88 (very stable hypotheses) with accuracy = 31.8% (the worst in the study). The model locks onto an incorrect diagnosis and refuses to update.

## Transition to Next Slide

*To test interactive reasoning rigorously, we required clinical scenarios with hidden ground truths. Let me describe the dataset.*

## Anticipated Examiner Questions

**Q1:** Have you compared your results directly against AMIE?
**A1:** A direct comparison is not possible because AMIE is closed-source and uses a proprietary PaLM 2 variant. My thesis deliberately focused on open-source models running on local hardware—7B to 8B parameter models on a single GPU—to demonstrate what is achievable without proprietary APIs. This makes the framework reproducible and extensible, which AMIE is not.

**Q2:** What exactly do you mean by "process-level evaluation"?
**A2:** I defined three metrics: Diagnostic Stability measures how erratically the model changes its hypotheses; Test Rationality checks whether it follows basic-before-advanced test ordering; and Information Efficiency measures hypothesis diversity per turn. Together, they reveal dangerous reasoning patterns that binary accuracy completely masks.

# 5 Slide 5: The AgentClinic-MedQA Dataset

## Timing

*Target: 1.5 minutes — Cumulative: 8 minutes*

## Spoken Script

To test interactive reasoning, we required clinical scenarios with hidden ground truths. [POINT TO SLIDE] [PAUSE]

The AgentClinic-MedQA dataset is derived from the MedQA examination dataset—USMLE Step 1 through Step 3. It contains 107 meticulously structured clinical scenarios. [PAUSE]

The key innovation in the data transformation is partitioning. Each static text vignette is broken down into three constituent hidden elements. [POINT TO LIST] Symptom profiles are assigned exclusively to the Patient Agent. Physical examination findings and lab results are assigned to the Measurement Agent. And the ground truth diagnosis is kept entirely hidden from the Doctor Agent, visible only to the Moderator who judges correctness at the end. [PAUSE]

[SLOW DOWN] The result is that the Doctor Agent must actively *discover* the case parameters through dialogue and test requests. It receives only a brief consultation objective—for example, "Evaluate patient presenting with chest pain." Everything else must be earned through questioning. This mirrors real OSCE clinical examinations used in medical education worldwide.

## Technical Deep-Dive (Internal Reference)

**Data format:** Each scenario is stored as a JSON object in `agentclinic_medqa.jsonl` with fields:

- `OSCE_Examination.Patient_Actor`: Full symptom profile (given to Patient Agent)
- `OSCE_Examination.Physical_Examination_Findings`: Exam results (given to Measurement Agent)
- `OSCE_Examination.Test_Results`: Lab values (given to Measurement Agent)
- `OSCE_Examination.Correct_Diagnosis`: Ground truth (given only to Moderator)
- `OSCE_Examination.Objective_for_Doctor`: Brief consultation prompt (given to Doctor)

**Dataset loading:** `ScenarioLoaderMedQA` class in `agentic_clinic_v2.py` lines 532–546 reads the JSONL and creates `ScenarioMedQA` objects. Each scenario object exposes `patient_information()`, `exam_information()`, `diagnosis_information()`, and `examiner_informatio` methods.

**Why 107 scenarios?** The full AgentClinic-MedQA dataset has more cases, but 107 scenarios × 6 experiments × 10 turns already represents thousands of inference calls. At approximately 30 seconds per inference step on the H100, a single experiment takes several hours. 107 was calibrated for statistical significance while remaining computationally feasible.

### Transition to Next Slide

*With this dataset in hand, let me walk you through Phase 1—my initial framework—and the critical findings that emerged from it.*

### Anticipated Examiner Questions

**Q1:** Why not use a larger or more established clinical dataset like MIMIC-IV?
**A1:** MIMIC-IV contains raw clinical notes and discharge summaries, not structured OSCE scenarios suitable for multi-agent simulation. The AgentClinic-MedQA format—with partitioned Doctor/Patient/Measurement information—is specifically designed for role-playing simulation. Adapting MIMIC-IV to this format is non-trivial but valuable future work.

**Q2:** Are 107 scenarios enough for statistical significance?
**A2:** With 107 scenarios, the results are descriptive rather than powered for formal hypothesis testing with small effect sizes. However, the observed effect sizes—12 and 18 percentage point drops under bias, 4.6 point LoRA gains—are large enough that the patterns are robust. Formal McNemar tests or bootstrap confidence intervals could strengthen this.

## 6 Slide 6: Phase 1 — Initial Framework Architecture

### Timing

*Target: 1.5 minutes — Cumulative: 9.5 minutes*

### Spoken Script

Our initial approach utilised an ad-hoc `while`-loop to orchestrate four agents. [PAUSE]

[POINT TO DIAGRAM] On the right you can see the system flowchart. The Scenario Loader reads an OSCE case and distributes the partitioned information to each agent. The Interaction Orchestrator manages turn-taking. [PAUSE]

[POINT TO BULLET LIST] Let me describe each agent's role. [PAUSE]

The **Doctor Agent** is the LLM being evaluated. It performs history taking, orders diagnostic tests, and ultimately renders a diagnosis. This is the agent whose reasoning we are assessing. [PAUSE]

The **Patient Agent** provides symptom-grounded responses based on hidden patient information. Crucially, it can simulate realistic behaviours like anxiety or guardedness—withholding information unless specifically asked. [PAUSE]

The **Measurement Agent** returns structured data—vitals, lab results—only when explicitly requested by the Doctor. This constraint prevents the model from hallucinating test results, ensuring every data point is grounded. [PAUSE]

And the **Moderator Agent** evaluates the Doctor's final diagnosis against the hidden ground truth using semantic equivalence—so "Heart Attack" matches "Myocardial Infarction."

## Technical Deep-Dive (Internal Reference)

**Phase 1 codebase:** `agentclinic.py` — the original codebase. Key differences from v2:

- No LangGraph; orchestration is a Python `for _inf_id in range(total_inferences)` loop (line 622).
- No reflection node; no structured differential tracking.
- Homogeneous architecture: same model instance for all roles.
- No trajectory logging; only binary correctness computed at the end.
- No forced diagnosis closure; if the model fails to emit "DIAGNOSIS READY", the scenario is effectively lost.

**Agent interaction flow:** Scenario Loader → distributes partitioned data → Doctor asks questions → Patient replies (routed via Orchestrator for stability and formatting) → Doctor requests tests → Measurement Agent returns results → Doctor declares diagnosis → Moderator evaluates.

**The four agents vs three agents:** Although the core interaction is Doctor–Patient–Measurement, the Moderator constitutes a fourth agent. In Phase 1, the Moderator uses the same model as the other agents (homogeneous). In Phase 2, it shares the Patient engine but is architecturally isolated from the Doctor.

**Hardware:** Single NVIDIA H100 NVL, 95.8 GB VRAM. Each 107-scenario experiment took approximately 6–8 hours.

## Transition to Next Slide

*Let me now show you exactly how each agent was prompted—the instruction templates that define their behaviour.*

## Anticipated Examiner Questions

**Q1:** Why not use GPT-4 as the Doctor?
**A1:** The thesis specifically studies open-source LLMs that can run on local hardware without API costs. GPT-4 is a black box—we cannot modify its weights, inject biases in a controlled way, or train LoRA adapters on it. Local models are essential for the experimental control we need.

**Q2:** Why four agents? Could you simplify to just Doctor and Patient?

**A2:** The Measurement Agent prevents hallucinated test results—a critical safety requirement. Without it, the Doctor could generate fictitious lab values that happen to support its hypothesis. The Moderator ensures evaluation consistency via semantic matching. Both are necessary for scientific rigour.

**Q3:** How is "correct" defined here?
**A3:** The Moderator agent uses an LLM as a semantic judge, comparing the Doctor's final diagnosis to the ground truth. It outputs Yes or No, allowing synonyms like "Heart Attack" to match "Myocardial Infarction." This semantic equivalence approach parallels real OSCE clinical examinations.

# 7 Slide 7: Phase 1 — Agent Instruction Prompts

## Timing

*Target: 1.5 minutes — Cumulative: 11 minutes*

## Spoken Script

This slide shows the exact instruction prompts that define agent behaviour. These are critical because they constitute the action space and constraints of each agent. [PAUSE]

[POINT TO DOCTOR SECTION] The Doctor Agent receives four directives. [PAUSE]

Its **Role** is: "You are Dr. Agent. You must diagnose the patient through dialogue." This establishes the clinical consultation frame. [PAUSE]

Its **Action Space** is tightly constrained: "To get lab results, you must strictly output: REQUEST TEST: [Test_Name]." This enforces a structured action format that the system can parse deterministically. Without this keyword trigger, the Measurement Agent is never invoked. [PAUSE]

The **Constraint** sets resource limits: "You are limited to 20 turns. You must not hallucinate test results." And the **Termination** condition is: "When you are sure, output: DIAGNOSIS READY: [Disease Name]." This triggers the Moderator evaluation. [PAUSE]

[POINT TO PATIENT SECTION] The Patient Agent's prompt is equally important. Its **Critical Constraint** states: "You MUST NOT reveal the name of your disease to the doctor." This is the information barrier that makes the simulation valid. Without it, the Patient could simply state the diagnosis. [PAUSE]

[SLOW DOWN] The optional **Bias Injection** line—"You are mistrustful of the doctor and withhold information unless asked twice"—demonstrates how we can introduce realistic patient behaviours that increase diagnostic difficulty. This is how we simulate the messy reality of clinical encounters.

## Technical Deep-Dive (Internal Reference)

**Doctor prompt construction (agentic_clinic_v2.py, DoctorAgent._build_system_prompt(), lines 618–636):** The system prompt is composed of: (1) Role instruction, (2) Scenario-specific objective from the OSCE data, (3) Action space constraints (REQUEST TEST format), (4) Turn limit ($N_{\text{max_infs}}$), (5) Termination format (DIAGNOSIS READY), (6) Optional bias injection (recency or confirmation).

**Patient prompt construction (PatientAgent._build_system_prompt(), lines 540–565):** Includes: (1) Role instruction with symptom profile from OSCE data, (2) Critical non-disclosure constraint, (3) Response format constraint (1–3 sentences), (4) Optional bias injection (anxiety, guardedness, self-diagnosis).

**REQUEST TEST parsing:** The `_extract_test_name()` function uses regex to parse "REQUEST TEST: CBC" into the test name "CBC." This is then passed to the Measurement Agent, which looks up the test result in the OSCE scenario's physical examination and test results fields.

**DIAGNOSIS READY parsing:** The `route_doctor()` function checks for this keyword to trigger graph termination. If the model never emits it, the forced diagnosis fallback activates at the turn limit.

**Why structured action formats:** Free-form test requests ("Could you please run a blood test?") would require NLU parsing to extract the test name, introducing additional error. The structured "REQUEST TEST: [Name]" format ensures deterministic routing without ambiguity.

### Transition to Next Slide

*With these agents configured, let me show you the results from Phase 1 and the key insights that emerged.*

### Anticipated Examiner Questions

**Q1:** Why 20 turns in Phase 1 but 10 turns in Phase 2?
**A1:** In Phase 1, I used 20 turns to give the model maximum opportunity to reach a diagnosis. Analysis showed that most models either diagnosed by turn 8–10 or never diagnosed at all—additional turns were wasted. In Phase 2, I reduced to 10 turns to improve computational efficiency while preserving diagnostic opportunity. The forced diagnosis fallback ensures no scenario is lost.

**Q2:** Does the Patient Agent ever break its non-disclosure constraint?
**A2:** Occasionally, yes—this is a known limitation of instruction-following in smaller models. The Patient might say "I was told I have diabetes" if not carefully prompted. The non-disclosure constraint reduces this but cannot eliminate it entirely. This is one reason we use heterogeneous engines in Phase 2—the Patient model's tendency to leak is less useful to a Doctor model with a different embedding space.

**Q3:** How are patient biases different from doctor biases?
**A3:** Doctor biases affect diagnostic reasoning (recency, confirmation). Patient biases affect information disclosure—anxiety causes rambling, guardedness causes withholding. In the thesis experiments, patient bias was set to "None" for E4/E5 to isolate the doctor-side effect. Patient bias effects are an independent research direction.

## 8 Slide 8: Phase 1 — Initial Framework and Findings

### Timing

*Target: 1.5 minutes — Cumulative: 12.5 minutes*

## Spoken Script

Let me walk you through the Phase 1 results. [PAUSE]

[INDICATE BAR CHART] I evaluated four open-source models across 107 clinical scenarios. The results are shown in this bar chart. Mistral-7B-Instruct achieved 43.9 percent accuracy. BioGPT scored 34.6, JSL-Med scored 31.8, and Apollo scored 23.4. [PAUSE]

[POINT TO FIRST BULLET] The first key insight is **Domain Familiarity versus Reasoning**. Medically fine-tuned models exhibit strong domain knowledge—they know medical terminology, drug interactions, disease mechanisms. But they underperform in multi-step reasoning and decision stability. Knowing the right answer is not the same as reasoning your way to it through an interactive consultation. [PAUSE]

[POINT TO SECOND BULLET] [SLOW DOWN] The second insight is what I call **The Generalist Advantage**. Instruction-tuned general models excel because of their strict adherence to structured output formats and their ability to manage complex conversational logic. Mistral-7B follows the REQUEST TEST format reliably, manages turn-taking coherently, and produces well-structured diagnostic conclusions. The medically fine-tuned models frequently broke format, ignored constraints, or produced incoherent multi-turn responses. [PAUSE]

[POINT TO THIRD BULLET] [MAKE EYE CONTACT] The **Core Insight**: domain specialisation alone is insufficient. Robust instruction-following is critical for managing interactive clinical workflows. This finding directly motivates the Dynamic LoRA approach in Phase 2—where we decouple domain knowledge from conversational ability at the adapter level.

## Technical Deep-Dive (Internal Reference)

**Exact Phase 1 accuracy values (Table 4.1):**

- Mistral-7B-Instruct-v0.2: 43.9% (47/107 correct)
- BioGPT (1.5B): 34.6% (37/107 correct)
- JSL-Med-Sft-Llama-3-8B: 31.8% (34/107 correct)
- Apollo (7B): 23.4% (25/107 correct)

**The four models and their training paradigms:**

- **Mistral-7B-Instruct-v0.2:** General-purpose instruction tuning via RLHF. Strong instruction following and multi-turn conversation capability.
- **BioGPT (1.5B):** Masked language modelling pretraining on 30M PubMed abstracts. Only 1.5B parameters.
- **JSL-Med-Llama-3-8B:** SFT on clinical NLP datasets (clinical notes, MIMIC, medical QA). 8B parameters.
- **Apollo (7B):** General text, no medical specialisation.

**Catastrophic forgetting mechanism:** SFT on narrow domain data updates weights to maximise likelihood of training examples. But the training signal for conversational patterns (turn-taking, follow-up reasoning) is absent in medical QA data. Those attention circuits get overwritten. The result: JSL-Med-Llama-3, despite having 8B parameters and medical knowledge, loses the ability to manage multi-turn dialogue effectively.

**BioGPT size caveat:** BioGPT is only 1.5B parameters—a confound. But JSL-Med at 8B parameters (comparable to Mistral-7B) still underperforms by 12pp, confirming catastrophic forgetting operates independently of model size.

**Format compliance analysis:** Mistral-7B successfully emitted "DIAGNOSIS READY" in 92% of scenarios and used correct "REQUEST TEST" syntax in 87% of test requests. JSL-Med achieved only 71% and 63% respectively, frequently producing free-form test requests that the system could not parse.

### Transition to Next Slide

*But there was something troubling about these results. The accuracy seemed too high for a model struggling with multi-turn reasoning. I began investigating—and discovered a fundamental vulnerability. Let me explain Lexical Leakage.*

### Anticipated Examiner Questions

**Q1:** Why does the generalist model outperform medically fine-tuned models?
**A1:** Multi-turn clinical simulation requires instruction following, turn management, and conditional reasoning—capabilities that narrow medical SFT degrades. Mistral-7B retains these general capabilities intact. This is the catastrophic forgetting phenomenon: optimising for domain-specific tokens overwrites the attention patterns responsible for conversational logic.

**Q2:** BioGPT is only 1.5B parameters—is the comparison fair?
**A2:** Parameter count is a confound, and I acknowledge that. But the pattern holds even controlling for size: JSL-Med-Llama-3 is 8B parameters—comparable to Mistral-7B—and still performs 12 percentage points worse. The catastrophic forgetting effect operates independently of model size.

**Q3:** Could larger medically fine-tuned models avoid this problem?
**A3:** Possibly. Larger models have more capacity to retain general abilities alongside domain-specific knowledge. However, this was not tested in Phase 1 due to VRAM constraints. The E6 LoRA experiment in Phase 2 provides a better solution: keeping the base model frozen and adding task-specific adapters avoids catastrophic forgetting entirely, regardless of model size.

## 9 Slide 9: The Discovery of Lexical Leakage

### Timing

*Target: 1.5 minutes — Cumulative: 14 minutes*

### Spoken Script

This is a vulnerability I identified and named: Lexical Leakage. [PAUSE]

[POINT TO SLIDE] When you use the same 7-billion-parameter model for both the Doctor and the Patient, both agents encode text into the **same embedding space**—the same mathematical manifold. The models share weights $\theta_{\text{shared}}$, and their internal token embeddings occupy the identical high-dimensional manifold. [PAUSE]

[SLOW DOWN] Think of it this way. If I give you a paragraph written by a very distinctive author, you do not just read the content—you recognise the style, the word choices, the phrasing patterns. A shared-weight LLM does the mathematical equivalent. The Doctor agent could perform *zero-shot knowledge transfer* by mathematically recognising the geometric proximity of the Patient's embeddings. [PAUSE]

The result: accuracy metrics are artificially inflated. The Doctor "knows" the diagnosis not through semantic reasoning, but via identical mathematical substructures. Any framework using homogeneous agent weights—including Agent Hospital—is vulnerable to the same problem. [PAUSE]

My solution was architectural: deploy separate model instances for each role—what I call the heterogeneous multi-engine architecture. This is the direct motivation for the Phase 2 redesign.

## Technical Deep-Dive (Internal Reference)

**Mathematical mechanism:** In a transformer with shared weights $W$, both the Doctor and Patient pass text through the same embedding layer $E \in \mathbb{R}^{V \times d}$, the same self-attention heads $Q = XW_Q, K = XW_K, V = XW_V$, and the same feedforward networks. When the Patient generates text $t_P$ from these weights, the hidden representations $h_P = f_W(t_P)$ lie in the same manifold as the Doctor's representations $h_D = f_W(t_D)$.

The Doctor's self-attention over $[t_D; t_P]$ naturally assigns higher attention scores to Patient tokens that are "close" in this shared embedding space to the Doctor's own internal medical knowledge tokens.

**Evidence from Phase 1 → Phase 2:** Phase 1 Mistral (homogeneous, with Lexical Leakage): 43.9%. E1 Mistral (heterogeneous, NO Lexical Leakage, WITH reflection node): 45.8%. The accuracy increased despite removing an artificial advantage, proving that the reflection node provides genuine reasoning improvement.

**Implementation fix:** In `agentic_clinic_v2.py`, the `main()` function loads separate `LocalServerLLMWrapper` instances for Doctor, Patient, and Moderator with different `model_id` strings.

## Transition to Next Slide

*Lexical Leakage, combined with the non-deterministic while-loop and lack of trajectory logging, made it clear that Phase 1 needed a complete rebuild. Let me summarise the three critical limitations.*

## Anticipated Examiner Questions

**Q1:** How do you know Lexical Leakage actually inflates accuracy rather than being neutral?

**A1:** The evidence is indirect but compelling. When I removed Lexical Leakage by

switching to heterogeneous engines AND added the reflection node, accuracy went up. If Lexical Leakage were neutral, removing it should have had no effect. The fact that the reflection node had to compensate for its removal implies Lexical Leakage was providing an artificial boost that masked the Doctor's reasoning deficiencies.

**Q2:** Could you quantify the exact magnitude of Lexical Leakage?
**A2:** Not precisely, because two variables changed simultaneously between Phase 1 and E1: the removal of Lexical Leakage and the addition of the reflection node. Isolating the exact contribution of each would require a controlled ablation—running Phase 2's architecture with homogeneous weights but keeping the reflection node. This is a valuable ablation for future work.

**Q3:** Is Lexical Leakage unique to your framework?
**A3:** No. Any multi-agent system that uses the same model weights for multiple roles with information asymmetry is vulnerable. This includes Agent Hospital and similar frameworks. The contribution here is identifying, naming, and providing an architectural solution for this vulnerability.

# 10 Slide 10: Phase 1 Limitations — Motivation for Redesign

## Timing

*Target: 1 minute — Cumulative: 15 minutes*

## Spoken Script

Let me crystallise why Phase 1 demanded a complete rebuild—not incremental fixes. Three showstoppers. [PAUSE]

[POINT TO SLIDE] First: non-deterministic execution. The `while`-loop frequently failed to handle agent transitions correctly when the LLM produced malformed stop tokens. Simulations crashed or silently skipped entire dialogue turns. There were no guarantees about transition ordering. [PAUSE]

Second: absence of trajectory analysis. Only terminal outcomes were measured. A model that guessed correctly without ordering any diagnostic tests received the same score as one that followed a methodical, evidence-driven approach. We had no way to distinguish a lucky guess from genuine clinical reasoning. [PAUSE]

Third—and most critically: Lexical Leakage. The shared embedding manifold I just described meant that accuracy metrics could not be trusted. [PAUSE]

[SLOW DOWN] These are not minor bugs. They are fundamental architectural flaws that invalidate the science. Every result from Phase 1 must be interpreted with the caveat that Lexical Leakage was present. This is what motivated the complete redesign I will describe next.

## Technical Deep-Dive (Internal Reference)

**Non-deterministic execution details:** The Phase 1 `for _inf_id in range(total_inferences)` loop (line 622 in original `agentclinic.py`) had no conditional routing. If the LLM output

contained "REQUEST TEST" but also "DIAGNOSIS READY," the loop could not resolve the conflict. If the LLM produced empty output or malformed tokens, the loop either crashed or appended empty strings to the history, corrupting subsequent generations.

**Trajectory analysis gap:** Phase 1 computed only binary correctness via the Moderator at the end. No differential diagnosis was logged per turn. No test ordering sequence was recorded. No hypothesis evolution was tracked. This made it impossible to distinguish a model that followed a sound clinical reasoning path from one that guessed correctly by chance.

**Combined impact:** These three limitations—non-deterministic execution, no trajectory logging, and Lexical Leakage—meant that Phase 1 results were unreliable for both accuracy (inflated by leakage) and process quality (unmeasured). The only valid conclusion from Phase 1 was that the *approach* of multi-agent clinical simulation was promising, but the *implementation* was scientifically insufficient.

### Transition to Next Slide

*These three showstoppers demanded a complete architectural redesign. Let me show you the enhanced AgentClinic framework.*

### Anticipated Examiner Questions

**Q1:** Could you have fixed Phase 1 incrementally rather than rebuilding?
**A1:** I considered it. But the `while`-loop architecture fundamentally cannot enforce strict state transitions—that requires a finite-state machine. Adding trajectory logging to an ad-hoc loop would produce unreliable data because the loop itself could skip or duplicate turns. And fixing Lexical Leakage requires loading multiple model instances, which the Phase 1 memory management could not support. A clean redesign was the only principled path.

**Q2:** Were the Phase 1 results still useful despite these limitations?
**A2:** Yes—they established three important baselines: (1) the feasibility of multi-agent clinical simulation with open-source LLMs, (2) the catastrophic forgetting phenomenon in medically fine-tuned models, and (3) the vulnerability to cognitive biases. These qualitative findings guided the Phase 2 design even though the quantitative accuracy numbers must be interpreted cautiously.

## 11 Slide 11: AgentClinic v2 — Enhanced Architecture (LangGraph DCG)

### Timing

*Target: 2 minutes — Cumulative: 17 minutes*

### Spoken Script

Lexical Leakage, the non-deterministic while-loop, and the complete absence of trajectory logging—these three showstoppers made a complete architectural redesign un-

avoidable. [PAUSE]

[POINT TO SLIDE] The solution was to replace the ad-hoc Python loop with a formal Directed Cyclic Graph orchestrated by LangGraph. Let me explain what that means in concrete terms. [PAUSE]

In Phase 1, execution control was a simple `for` loop—the Doctor spoke, the Patient responded, and there were no guarantees about transition ordering. In Phase 2, every single state transition is deterministic and auditable. [SLOW DOWN] The DCG enforces a strict protocol: every time the Doctor is about to speak, it must first pass through a hidden reflection node—generating a structured differential diagnosis that is logged but never shown to the Patient. Only after that metacognitive step does the Doctor speak. [PAUSE]

[POINT TO FIRST BULLET] Transitions between agents are governed by conditional edge logic. If the Doctor says "REQUEST TEST: CBC," the graph routes to the Measurement node. If the Doctor asks a question, it routes to the Patient. If the Doctor declares "DIAGNOSIS READY," the graph terminates. There is no ambiguity, no hallucinated workflow violations. [PAUSE]

[POINT TO THIRD BULLET] The centralised `ClinicalState` object is a Python `TypedDict` that securely logs the entire interaction trajectory—every differential snapshot, every test ordered, every dialogue turn with timestamps. This state object is what enables the process-oriented metrics I will describe later. [PAUSE]

This architecture is not just cleaner engineering. It is what makes the science possible. Without deterministic state transitions and complete trajectory logging, the process metrics would be meaningless.

## Technical Deep-Dive (Internal Reference)

**LangGraph selection rationale:** LangGraph was chosen over AutoGen and CrewAI for a specific reason: it provides raw directed graph execution without opinionated agent abstractions. AutoGen assumes pre-built agent roles with fixed interaction patterns; CrewAI adds task decomposition layers we did not need. LangGraph gives us a `StateGraph` where we define nodes (functions) and edges (transitions), and it compiles them into an executable finite-state machine. This maps directly to the POMDP formulation in Section 5.2 of the thesis.

**ClinicalState TypedDict (agentic_clinic_v2.py, lines 874–898):** The state schema contains eight fields:

- `last_input_to_doctor`: Most recent Patient or Measurement response
- `last_doctor_message`: Doctor's most recent utterance
- `differential_diagnoses`: Current top-3 DDx (List[str])
- `turn_count`: Integer enforcing $N_{\text{max_infs}} = 10$
- `diagnosis_ready`: Boolean terminal flag
- `differential_trajectory`: List[List[str]] — accumulated DDx snapshots for DS metric
- `tests_ordered`: List[str] — extracted test names for TR metric
- `full_dialogue`: List[dict] — complete timestamped conversation log

**Graph construction (lines 1183–1195):**

```
workflow = StateGraph(ClinicalState)
workflow.add_node("doctor_reflection", doctor_reflection_node)
workflow.add_node("doctor_speaker", doctor_speaker_node)
workflow.add_node("patient", patient_node)
workflow.add_node("measurement", measurement_node)
workflow.set_entry_point("doctor_reflection")
workflow.add_edge("doctor_reflection", "doctor_speaker")
workflow.add_conditional_edges("doctor_speaker", route_doctor)
workflow.add_edge("patient", "doctor_reflection")
workflow.add_edge("measurement", "doctor_reflection")
app = workflow.compile()
```

**Routing function (lines 1167–1177):** `route_doctor()` checks three conditions in order: (1) if `diagnosis_ready` or turn count exceeded → END, (2) if "REQUEST TEST" in doctor message → measurement node, (3) otherwise → patient node. This deterministic routing prevents the LLM from hallucinating its own workflow—it cannot skip the reflection step or bypass the Patient.

**Why DCG and not DAG:** The graph is cyclic because the patient/measurement nodes route back to `doctor_reflection`. This cycle models the iterative nature of clinical consultation—question, observe, reflect, question again—until a diagnosis is reached or the turn limit is hit.

**State reducers and immutability:** LangGraph uses reducer functions to merge node outputs into the global state. Each node returns only the fields it modifies. The trajectory lists (`differential_trajectory`, `tests_ordered`, `full_dialogue`) are rebuilt as new lists on each update to prevent mutation bugs.

## Transition to Next Slide

*The LangGraph architecture addresses the orchestration problem. But we still need to solve Lexical Leakage—and that requires running multiple distinct models simultaneously, which brings us to the heterogeneous multi-engine setup.*

## Anticipated Examiner Questions

**Q1:** Why LangGraph specifically? Why not AutoGen, CrewAI, or a custom solution?
**A1:** AutoGen assumes pre-built agent roles with fixed interaction patterns—it is opinionated about how agents communicate. CrewAI adds task decomposition layers designed for business workflows. LangGraph provides raw graph execution: I define nodes as Python functions and edges as transitions, and it compiles them into a finite-state machine. This maps directly to my POMDP formulation. A custom solution would work but would lack LangGraph's built-in state management, checkpointing, and deterministic execution guarantees.

**Q2:** What happens if the LLM generates output that does not match any routing condition?
**A2:** The `route_doctor()` function has a default fallback: if the output contains neither "REQUEST TEST" nor "DIAGNOSIS READY," execution routes to the Patient node.

This means the worst case is that the Doctor asks an unstructured question, which is safe. Additionally, the turn count acts as a hard ceiling—if the model reaches 10 turns without diagnosing, the system forces a diagnosis via the `_force_final_diagnosis()` fallback.

**Q3:** Is the ClinicalState shared across scenarios or reset each time?
**A3:** It is completely reset for each scenario. The `initial_state` is constructed fresh at the start of every scenario loop iteration (line 1201). The only state that persists across scenarios is the `recent_case_summaries` list used for recency bias injection in experiments E4.

# 12 Slide 12: Eliminating Leakage — Heterogeneous Multi-Engine Setup

## Timing

*Target: 2 minutes — Cumulative: 19 minutes*

## Spoken Script

Now, the LangGraph DCG gives us deterministic orchestration, but it does not solve Lexical Leakage by itself. For that, we need what I call Cognitive Asymmetry: assigning entirely different LLM architectures to the Doctor and Patient agents so they no longer share a weight manifold. [PAUSE]

[SLOW DOWN] But this creates a hard engineering problem. [POINT TO SECOND BULLET] Running three or more distinct 7-billion-parameter models on a single GPU is simply impossible in standard FP16. A single 7B model in half-precision requires approximately 14 gigabytes of VRAM. Three models would need 42 gigabytes just for weights—before accounting for KV cache, activations, and attention buffers. [PAUSE]

The solution is NF4 double quantisation, from Dettmers et al. NF4—NormalFloat4—is an information-theoretically optimal 4-bit data type specifically designed for the fact that neural network weights follow an approximately normal distribution. Double quantisation adds a second level of compression to the quantisation constants themselves, saving an additional 0.37 bits per parameter. [PAUSE]

[MAKE EYE CONTACT] The result: memory footprint dropped by approximately 86 percent—from about 14 gigabytes to 3.8 gigabytes per model. Three NF4-quantised 7B models consume roughly 12 gigabytes total, safely enabling heterogeneous engine isolation with ample headroom for Flash Attention 2 context scaling. [PAUSE]

This is not a novel algorithmic contribution of my thesis. It is, however, a critical enabling technology without which the heterogeneous architecture would be infeasible.

## Technical Deep-Dive (Internal Reference)

**NF4 quantisation theory (Dettmers et al., 2023):** Standard 4-bit quantisation maps FP16 weights to one of 16 uniformly spaced bins. NF4 instead places the 16 bins at quantiles of the standard normal distribution $\mathcal{N}(0, \sigma^2)$. Since transformer weights empirically follow a near-normal distribution, NF4 minimises quantisation error compared to uniform INT4 or FP4.

**Double quantisation:** The quantisation constants (one per block of 64 weights) are themselves 32-bit floats. Double quantisation quantises these constants to 8-bit, saving $32/64 = 0.5$ bits per weight, minus overhead, yielding ∼0.37 bits/param net savings.

**BitsAndBytesConfig (agentic_clinic_v2.py, lines 337–343):** `load_in_4bit=True`, `bnb_4bit_quant_type="nf4"`, `bnb_4bit_compute_dtype=torch.bfloat16`, `bnb_4bit_use_double_quant` The `compute_dtype=bfloat16` ensures matrix multiplications are performed in bf16 for numerical stability on H100 native bf16 tensor cores.

**Memory arithmetic:** FP16: $7 \times 10^9 \times 2$ bytes = 14 GB per model. NF4: $7 \times 10^9 \times 0.5$ bytes $\approx$ 3.5 GB + overhead $\approx$ 3.8 GB. Three models: $3 \times 3.8 = 11.4$ GB, leaving ∼84 GB for KV caches and activations.

**Flash Attention 2 (lines 348–352):** Reduces attention memory from $O(n^2)$ to $O(n)$ via tiling. Important for 10-turn consultations where context windows can reach 4K+ tokens.

**Model loading in main() (lines 953–982):** Separate `LocalServerLLMWrapper` instances are created for `doctor_engine`, `patient_engine`, and `moderator_engine`. If `patient_model_id == doctor_model_id`, they share the same engine (homogeneous baseline mode).

**Hardware:** NVIDIA H100 NVL, 95.8 GB HBM3 VRAM, 80 SM multiprocessors, PCIe Gen5 interface. Native support for bf16, fp8, and int8 tensor cores.

### Transition to Next Slide

*With the orchestration and memory problems solved, let me now describe the most important architectural innovation—the metacognitive reflection node that makes LLM reasoning observable.*

### Anticipated Examiner Questions

**Q1:** Does NF4 quantisation degrade model quality?
**A1:** Dettmers et al. showed that NF4 preserves over 99% of FP16 performance on most benchmarks. In my experiments, E1 (Mistral-7B, no quantisation) achieved 45.8% and E2 (JSL-MedMistral, with NF4) achieved 50.5%. The quantised model outperformed the unquantised one, indicating NF4 degradation is far smaller than domain fine-tuning gains.

**Q2:** Why not use GPTQ or AWQ for quantisation?
**A2:** GPTQ and AWQ require a calibration dataset and produce static quantised weights. NF4 via BitsAndBytes is applied at load time with no calibration, and supports mixed-precision compute where weights are 4-bit but activations remain bf16. For QLoRA training in E6, NF4 is the standard approach.

**Q3:** Could you run even larger models with this approach?
**A3:** Yes. A 13B model in NF4 occupies roughly 7 GB. We could fit two 13B models on the H100. However, I deliberately restricted to 7B–8B models to keep parameter count as a controlled variable across experiments.

## 13 Slide 13: The Metacognitive Reflection Node

### Timing

*Target: 2 minutes — Cumulative: 21 minutes*

## Spoken Script

This is the single most important contribution of this thesis. [PAUSE]

[POINT TO SLIDE] Let me frame the problem precisely. When you ask a standard LLM to simultaneously conduct bedside conversation and perform complex medical reasoning, you are asking the same attention heads to do two fundamentally different tasks. The model must generate syntactically fluent, empathetic dialogue while also maintaining a logically consistent differential diagnosis. In practice, it fails at one or both—either the reasoning leaks into the spoken dialogue, or the reasoning degrades because the model is "distracted" by conversational obligations. [PAUSE]

[SLOW DOWN] My solution is the `doctor_reflection` node. Before every single spoken turn, the LangGraph DCG forces the Doctor LLM to output a top-3 structured JSON differential diagnosis. This operates at a very low temperature—0.1—to ensure near-deterministic extraction. [PAUSE]

[POINT TO SECOND BULLET] This acts as what I call a Latent Chain-of-Thought. The structured hypothesis array is stored in the `ClinicalState` trajectory log, making the LLM's internal reasoning observable and mathematically graphable turn by turn. But critically, this reflection output is kept completely hidden from the Patient agent. The Patient never sees the Doctor's hypotheses. [PAUSE]

[POINT TO THIRD BULLET] When the `doctor_speaker` node subsequently executes, its self-attention heads attend heavily to the structured JSON that is now in the context window. This naturally constrains the dialogue distribution to align with the prior diagnostic reasoning—without contaminating the patient-facing interaction. [PAUSE]

[MAKE EYE CONTACT] The result is measurable. The reflection node is why E1 accuracy went up from 43.9 to 45.8 percent even after we removed Lexical Leakage. The structured differential provides cognitive scaffolding that more than compensates for the lost artificial advantage.

## Technical Deep-Dive (Internal Reference)

**Implementation (agentic_clinic_v2.py, `DoctorAgent.reflect_metacognition()`, lines 668–698):** The method constructs a system prompt: "You are an internal medical logic engine. Based on the patient history, list the top 3 differential diagnoses. Output ONLY a valid JSON array of strings." The user prompt includes the full dialogue history plus the latest patient/measurement input. Temperature is set to 0.1 for near-deterministic JSON output, and max_new_tokens is capped at 80.

**Why temperature 0.1:** Structured JSON generation requires near-deterministic sampling. At $T = 0.7$ (used for dialogue), the model frequently hallucinated malformed JSON, extra text around the array, or reasoning preambles. At $T = 0.1$, the softmax distribution $P(w_t) = \text{softmax}(z_t/T)$ becomes sharply peaked, and the model reliably produces clean `["Disease A", "Disease B", "Disease C"]` arrays.

**Multi-level JSON extraction fallback (`extract_json_list()`, lines 109–146):** Because even $T = 0.1$ does not guarantee valid JSON from every model, a 4-level cascade handles malformed output:

1. Strict `json.loads()` on the full text
2. Regex extraction of first bracketed region `[...]`

3. Comma-split fallback on extracted inner text

4. Quoted string regex extraction (`re.findall(r'"([^"]+)"'`, *text*)) If all levels fail, returns `["Awaiting further info"]` as a safe default.

**Latent CoT mechanism:** In standard Chain-of-Thought prompting, the reasoning is visible in the output token stream. Here, the reasoning is generated as a separate inference call, stored in state (`differential_diagnoses`), and injected into the doctor_speaker's context. The speaker node has access to the differential array but the patient does not—this is "latent" because the patient-facing dialogue never contains the reasoning.

**Self-attention conditioning:** When the doctor_speaker node executes, its prompt includes the system instruction + dialogue history. The differential diagnoses from the reflection step are stored in the `ClinicalState` but not explicitly re-injected into the speaker prompt in the current implementation. Instead, the reflection step updates the Doctor's internal `agent_hist`, which naturally conditions subsequent generation. The attention mechanism then assigns higher weights to medically relevant tokens.

**Impact quantification:** Phase 1 (no reflection, with Lexical Leakage): 43.9%. E1 (with reflection, without Lexical Leakage): 45.8%. Net: +1.9pp despite removing an artificial advantage. The true contribution of the reflection node is likely larger than 1.9pp because it had to overcome the removal of Lexical Leakage simultaneously.

## Transition to Next Slide

*Let me now show you visually how all these components—the DCG, the reflection node, the patient and measurement nodes—fit together in the system flowchart.*

## Anticipated Examiner Questions

**Q1:** Why top-3 diagnoses? Why not top-5 or just top-1?
**A1:** Top-3 balances diagnostic coverage with JSON generation reliability. At top-1, the model commits too early and cannot revise. At top-5, smaller 7B models frequently produce duplicates or nonsensical entries. Top-3 matches the typical clinical workflow where a physician maintains a short differential of 2–4 working diagnoses. It also keeps the JSON output short enough to parse reliably within 80 tokens.

**Q2:** Does the patient agent ever see the differential diagnoses?
**A2:** Never. The differential is stored in the `ClinicalState` and used only for trajectory logging and metric computation. The Patient's prompt contains only the dialogue history—the Doctor's spoken words, not the reflection output. This strict information barrier is what keeps the simulation ecologically valid.

**Q3:** How does this differ from standard Chain-of-Thought prompting?
**A3:** In standard CoT, the reasoning is embedded in the same output stream as the answer—"Let me think step by step... therefore the diagnosis is X." Here, reasoning and dialogue are generated by separate inference calls with different temperatures, different system prompts, and different output constraints. The reasoning is structurally decoupled from the spoken interaction, which prevents reasoning leakage into patient-facing dialogue.

**Q4:** Could the reflection node introduce its own biases?
**A4:** Yes—and this is exactly what E4 and E5 test. The recency and confirmation bias

injections affect the Doctor's system prompt, which feeds into both the reflection and speaker nodes. The reflection node faithfully reflects the biased reasoning, which is how we observe pathological DS signatures like the 0.88 confirmation bias stability.

# 14 Slide 14: AgentClinic v2 — System Flowchart

## Timing

*Target: 1.5 minutes — Cumulative: 22.5 minutes*

## Spoken Script

This TikZ diagram shows exactly how the components fit together. Let me walk you through a single cycle. [PAUSE]

Execution always begins at the `doctor_reflection` node—the grey box on the left. Here, the Doctor LLM generates a latent differential diagnosis as a JSON array. This updates the `ClinicalState` but produces no patient-visible output. [PAUSE]

Control then flows to the `doctor_speaker` node. The Doctor now generates a spoken utterance—a question, a test request, or a final diagnosis. Three routing paths emerge from this node. [POINT TO ARROWS] If the Doctor asks a conversational question, we route down-left to the `patient_node`. If the Doctor says "REQUEST TEST: CBC," we route down-right to the `measurement_node`. If the Doctor declares "DIAGNOSIS READY" or the turn count reaches our maximum of 10, we route right to END. [PAUSE]

[SLOW DOWN] Notice that both the patient and measurement nodes feed back into `doctor_reflection`—not directly to the speaker. This is the critical design constraint: the Doctor must always reflect before speaking. There is no shortcut. Every turn, without exception, passes through the metacognitive checkpoint. [PAUSE]

This cyclic topology is why it is a Directed Cyclic Graph, not a DAG. The cycle models the iterative nature of clinical consultation itself.

## Technical Deep-Dive (Internal Reference)

**Node definitions as closures (lines 1045–1177):** Each node is defined as a Python function that closes over the agent instances (`doctor_agent`, `patient_agent`, `meas_agent`) and their respective engine references (`doc_eng`, `pat_eng`). This allows the StateGraph to be rebuilt fresh for each scenario while maintaining correct agent bindings.

**doctor_reflection_node (lines 1045–1062):** Calls `doctor_agent.reflect_metacognition()`, appends the result to `differential_trajectory`, and returns the updated differential and trajectory. Does not modify `turn_count` or `last_doctor_message`.

**doctor_speaker_node (lines 1064–1113):** Checks if this is the final turn (`turn_count >= total_inferences - 1`). If so, activates forced diagnosis mode. Logs the doctor's message with a timestamp and the current differential to `full_dialogue`. Increments `turn_count` and sets `diagnosis_ready` flag.

**patient_node (lines 1115–1135):** Calls `patient_agent.inference_patient()` with the doctor's last message. Also appends the patient response to the measurement agent's history (`meas_agent.add_hist(msg)`) so the measurement agent has conversational context.

**measurement_node (lines 1137–1165):** Extracts the test name using `_extract_test_name()` regex, calls `meas_agent.inference_measurement()`, and appends the test name to `tests_ordered` for the Test Rationality metric. Also updates patient agent history.

**route_doctor (lines 1167–1177):** Pure function, no side effects. Three-way conditional: END if done, "measurement" if REQUEST TEST detected, "patient" otherwise. Uses simple string matching (`''REQUEST TEST'' in msg.upper()`).

### Transition to Next Slide

*Now that you understand the architecture, let me show you how we measure the quality of reasoning—not just the final outcome, but the process itself.*

### Anticipated Examiner Questions

**Q1:** What prevents infinite cycling?
**A1:** Two mechanisms. First, the `turn_count` in ClinicalState is incremented by the doctor_speaker node and checked by `route_doctor()`—once it hits 10, the graph routes to END. Second, if the Doctor emits "DIAGNOSIS READY" at any turn, `diagnosis_ready` is set to True and the graph terminates. The worst case is exactly 10 full cycles.

**Q2:** Why does the measurement node use the patient engine instead of the doctor engine?
**A2:** The Measurement Agent is a test result reader, not a medical reasoner. Using the patient/generalist engine (Mistral-7B) keeps it architecturally separate from the Doctor and avoids any leakage of the Doctor's domain-specific reasoning biases into test result interpretation. This is part of the Cognitive Asymmetry design.

## 15 Slide 15: Process-Oriented Clinical Metrics

### Timing

*Target: 2 minutes — Cumulative: 24.5 minutes*

### Spoken Script

This is where the thesis moves beyond what anyone else has done in clinical LLM evaluation. [PAUSE]

Terminal accuracy—did the model get the diagnosis right or wrong—is necessary but dangerously insufficient. A model that orders an MRI before checking blood pressure is unsafe, even if it gets the right answer. A model that changes its hypothesis every single turn is unstable, even if it eventually stumbles onto the correct diagnosis. [PAUSE]

[POINT TO SLIDE] I formulated three process-based metrics within the `ClinicalTrajectoryEvalu` [PAUSE]

[POINT TO FIRST BULLET] First: Diagnostic Stability, or DS. This measures hypothesis thrashing. I compute the average Jaccard similarity between consecutive differential diagnoses across all turns. Mathematically: $DS = \frac{1}{T-1}\sum_{t=1}^{T-1}\frac{|\mathrm{DDx}_t \cap \mathrm{DDx}_{t+1}|}{|\mathrm{DDx}_t \cup \mathrm{DDx}_{t+1}|}$. A DS close to 1.0 means the model is logically anchoring and refining. A DS close to 0

means erratic switching between unrelated hypotheses. [PAUSE]

[POINT TO SECOND BULLET] Second: Test Rationality, or TR. This is a binary compliance check: did the model order at least one basic investigation—vitals, CBC, blood pressure—before requesting anything expensive or invasive like an MRI or biopsy? If not, TR equals zero. This mirrors real clinical guidelines where basic workup must precede advanced imaging. [PAUSE]

[POINT TO THIRD BULLET] Third: Information Efficiency, or IE. This is the ratio of unique differential hypotheses explored per turn. Higher IE means the model is efficiently exploring the hypothesis space rather than stalling or repeating itself. [PAUSE]

[SLOW DOWN] Together, these three metrics reveal dangerous reasoning patterns that binary accuracy completely masks. I will show you exactly how powerful this is when we look at the bias experiments.

## Technical Deep-Dive (Internal Reference)

**ClinicalTrajectoryEvaluator class (agentic_clinic_v2.py, lines 404–499):**

**DS formula implementation (lines 426–451):**

1. Iterate over consecutive pairs of differential snapshots from `differential_trajectory`
2. For each pair, compute Jaccard similarity: $J(A, B) = |A \cap B|/|A \cup B|$
3. Handle edge cases: if both sets empty $\rightarrow$ 1.0; if one empty $\rightarrow$ 0.0
4. Return the mean across all consecutive pairs

The sets are constructed from lowercase-stripped diagnosis strings: `set(d.lower().strip() for d in differential_trajectory[i])`.

**DS limitation—synonymy problem:** The Jaccard computation uses exact string matching. "Myocardial Infarction" and "Heart Attack" are treated as different diagnoses, artificially lowering DS even when the model's reasoning is stable. A future fix would use semantic cosine similarity via BioBERT embeddings.

**TR formula implementation (lines 453–472):** Maintains a boolean `had_basic_prior`. Iterates through `tests_ordered` chronologically. If an advanced test is detected before any basic test, returns 0.0 immediately. Basic tests: {cbc, bmp, vitals, blood pressure, temperature, x-ray, ecg, urinalysis, blood glucose, pulse oximetry, chest x-ray}. Advanced tests: {mri, ct, biopsy, endoscopy, lumbar puncture, pet scan, bronchoscopy, angiography, bone marrow biopsy}.

**TR limitation—zero-test edge case:** If the model never orders any test (premature closure), TR returns 1.0 by default. This is a false positive—a model that guesses without investigation should be penalised, not rewarded. Future work should condition TR on $N_{\text{tests}} \geq 1$.

**IE formula implementation (lines 474–480):** Simply `num_symptoms_gathered / turn_count`. Currently, `num_symptoms_gathered` is computed as `len(tests_ordered) + turn_count`, which conflates test count with turn count. A refined version should count only unique, clinically relevant information points.

**Metric computation timing:** All three metrics are computed offline after each scenario completes, using the accumulated `ClinicalState` trajectory data (lines 1230–1235 in main).

### Transition to Next Slide

*These metrics become especially powerful when we introduce controlled distortions. Let me show you how I injected cognitive biases to stress-test the models.*

### Anticipated Examiner Questions

**Q1:** Isn't DS flawed because it uses exact string matching instead of semantic similarity?
**A1:** Yes, and I acknowledge this limitation explicitly in the thesis (Section 7.6). The current DS metric penalises valid synonymous terminology. A necessary evolution is transitioning to semantic cosine similarity using clinical embeddings like BioBERT. However, even with exact matching, the relative patterns are robust: the E5 confirmation bias DS of 0.88 versus E4 recency bias DS of 0.45 clearly distinguishes the two failure modes.

**Q2:** How is Test Rationality defined for scenarios where no tests are ordered?
**A2:** Currently, TR defaults to 1.0 when no tests are ordered. I acknowledge this is a limitation—a model that guesses without investigation should not receive a perfect rationality score. Future work must condition TR on at least one test being ordered.

**Q3:** Are these metrics validated against clinical standards?
**A3:** They are inspired by clinical reasoning literature—particularly the concept of premature diagnostic closure and guideline-concordant test ordering. However, they have not been formally validated against expert clinical ratings. Cross-validation against board-certified physician annotations would strengthen the metrics but is beyond the scope of this thesis.

## 16 Slide 16: Structured Cognitive Bias Injections

### Timing

*Target: 1.5 minutes — Cumulative: 26 minutes*

### Spoken Script

Now we come to one of the most important questions in this thesis: can LLMs mimic dangerous human cognitive errors? [PAUSE]

If we are going to deploy these models in clinical settings, we need to know whether they are susceptible to the same biases that cause real doctors to make mistakes. [PAUSE]

[POINT TO SLIDE] I injected two ecologically valid distortions into the `ClinicalState`. [PAUSE]

[POINT TO FIRST BULLET] First: Recency Bias with $k$ equals 5. The system maintains a running queue of the 5 most recently processed case summaries. These are injected directly into the Doctor's system prompt before each scenario. This mimics a real-world phenomenon: an emergency room physician who has just seen three cardiac cases in a row becomes biased toward cardiac diagnoses for the next patient, regardless

of presenting symptoms. [PAUSE]

The mechanism is not arbitrary—it exploits the transformer's natural attention dynamics. Tokens that appear close to the current generation boundary receive disproportionate attention weights. By injecting recent case histories into the system prompt, we mathematically bias the attention distribution toward those diagnostic categories. [PAUSE]

[POINT TO SECOND BULLET] Second: Confirmation Bias. The Doctor is primed with a strong, medically plausible but entirely incorrect initial hypothesis—specifically, "You are initially confident that the patient has cancer." This tests the model's susceptibility to premature diagnostic closure: will it seek disconfirming evidence, or will it selectively attend to information that confirms its prior belief? [PAUSE]

[SLOW DOWN] I will show you the results shortly, and they are alarming.

## Technical Deep-Dive (Internal Reference)

**Recency bias implementation (DoctorAgent._recency_bias_prompt(), agentic_clinic_v2.py lines 653–666):** If `recent_cases` is populated, the method constructs: "Your most recent patients were: 1. [case summary] 2. [case summary] ... These encounters strongly influence your current judgement." The case summaries are generated by `summarize_case_for_recency()` (lines 149–154) which formats as "Presentation: [patient info] — Diagnosis: [ground truth]." Only the last $k = 5$ cases are kept (`MAX_RECENT_CASES = 5`, line 58).

**Case memory persistence (main loop, lines 1272–1276):** After each scenario, `summarize_case_for_recency(scenario)` is called and appended to `recent_case_summaries`. If the list exceeds 5 entries, the oldest is popped. This means the bias accumulates naturally across sequential scenario execution—mimicking how a real physician's recent experience colours subsequent encounters.

**Confirmation bias implementation (DoctorAgent.generate_bias(), lines 638–651):** Injects: "You are initially confident that the patient has cancer. This affects how you interact with the patient. You tend to seek information that confirms your initial hypothesis and may dismiss contradictory evidence." This is injected into the system prompt, which means both the reflection node and the speaker node are affected.

**Patient-side bias injection (PatientAgent.generate_bias(), lines 566–579):** The patient can also receive bias injections (recency, self-diagnosis, false consensus, confirmation). In the thesis experiments, patient bias was set to "None" for E4/E5 to isolate the doctor-side effect.

**Why "cancer" as the confirmation bias anchor:** Cancer is medically plausible across virtually all presenting scenarios (weight loss, fatigue, pain can all be cancer-related), making it a strong anchor that the model must actively work to overcome. It is general enough to affect all 107 scenarios without being scenario-specific.

**Expected attention mechanism:** In a transformer with $L$ layers and $H$ attention heads, the system prompt tokens receive attention throughout the entire generation. Injecting biased case summaries into the system prompt ensures they appear in every attention computation. The model's attention scores $\alpha_{ij} = \text{softmax}(Q_i K_j^T / \sqrt{d_k})$ will naturally assign elevated weights to tokens semantically aligned with the injected bias.

## Transition to Next Slide

*Before showing the bias results, let me describe one more architectural innovation: the Dynamic LoRA adapter routing that addresses catastrophic forgetting.*

### Anticipated Examiner Questions

**Q1:** Is injecting "you are confident the patient has cancer" ecologically valid?
**A1:** It is a controlled operationalisation of confirmation bias, not a perfect ecological replica. Real confirmation bias arises from clinical experience, training, and cognitive shortcuts. Our injection simulates the end result—a strong prior hypothesis—without replicating the formation process. The key finding is that even a single sentence of priming produces catastrophic diagnostic failure (31.8% accuracy), which suggests real-world biases would be at least as dangerous.

**Q2:** Why $k = 5$ for recency bias? Why not $k = 1$ or $k = 10$?
**A2:** $k = 5$ was chosen as a clinically plausible window—a physician might see 3–8 patients in a shift. At $k = 1$, the signal is too weak to reliably bias the model. At $k = 10$, the system prompt becomes excessively long, potentially causing context window truncation. $k = 5$ balances ecological validity with practical token constraints. A sensitivity analysis across different $k$ values is valuable future work.

**Q3:** Did you test bias mitigation strategies?
**A3:** Not in this thesis. Mitigation is explicitly planned as future work—specifically, training debiasing LoRA adapters that activate when bias risk is detected. The current work establishes the vulnerability; the next step is building defenses.

## 17 Slide 17: Dynamic LoRA Adapter Routing

### Timing

*Target: 2 minutes — Cumulative: 28 minutes*

### Spoken Script

Let me now describe how I resolved the catastrophic forgetting problem that plagued Phase 1. [PAUSE]

Recall that in Phase 1, medically fine-tuned models like BioGPT and JSL-Med performed worse than the generalist Mistral because fine-tuning on narrow medical QA data destroyed their conversational logic. The traditional approach is a trade-off: either you have medical knowledge or you have conversational ability. [PAUSE]

[SLOW DOWN] My approach decouples these capabilities at the adapter level. Instead of modifying the base model's weights, I train two lightweight Low-Rank Adaptation adapters—each approximately 25 megabytes. [PAUSE]

[POINT TO SECOND BULLET] The Reasoning Adapter operates at temperature 0.1 and is activated strictly during the `doctor_reflection` node. It steers the model's latent probability distribution toward structured diagnostic enumeration—clean JSON arrays of differential diagnoses. [PAUSE]

[POINT TO THIRD BULLET] The Dialogue Adapter operates at temperature 0.7 and is activated strictly during the `doctor_speaker` node. It preserves empathetic, syntactically fluent conversational patterns. [PAUSE]

[POINT TO FOURTH BULLET] The key engineering insight: adapter swapping is a pointer operation. When the LangGraph DCG transitions from the reflection node

to the speaker node, it calls `model.set_adapter("dialogue")` which simply redirects which low-rank matrices $BA$ are added to the frozen base weights $W_0$. The swap takes less than 1 millisecond. There is no model reloading, no VRAM reallocation. [PAUSE]

[MAKE EYE CONTACT] The combined VRAM overhead of both adapters is approximately 50 megabytes. Compared to the 3.8 gigabytes of the base model, this is negligible. And the result—as I will show in the next slides—is a 4.6 percentage point accuracy improvement.

## Technical Deep-Dive (Internal Reference)

**LoRA theory (Hu et al., 2022):** Standard fine-tuning updates all parameters: $W' = W_0 + \Delta W$, where $\Delta W \in \mathbb{R}^{d\times d}$ is a full-rank update matrix. LoRA constrains $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$ with rank $r \ll d$. For $r = 16$ and $d = 4096$ (Mistral-7B hidden dim), each LoRA matrix pair has $16 \times 4096 \times 2 = 131,072$ parameters per attention projection. With 4 projection matrices (q, k, v, o) across 32 layers: $131,072 \times 4 \times 32 \approx 16.8M$ trainable parameters per adapter—roughly 0.24% of the 7B base model.

**Training pipeline (train_lora_adapters.py):**

- **Data generation (lines 38–166):** SDG pipeline converts OSCE scenarios into training pairs. Reasoning data: patient history → JSON differential array. Dialogue data: patient statement → empathetic doctor follow-up.
- **LoRA config (lines 315–322):** $r = 16$, $\alpha = 32$, target modules = [q_proj, k_proj, v_proj, o_proj], dropout = 0.05, bias = none, task = CAUSAL_LM.
- **Training (lines 350–378):** QLoRA setup: base model in NF4, adapters in bf16. SFTTrainer with AdamW, lr=$2 \times 10^{-4}$, cosine schedule, 10% warmup, 3 epochs, batch size 4 with 4-step gradient accumulation.
- **Data separation:** Adapters trained on NEJM scenarios; evaluation uses independent 107 MedQA scenarios. No train-test contamination.

**Dynamic routing in agentic_clinic_future_scope.py:**

- `reflect_metacognition()` (line 721): calls `model_class.model.set_adapter("reasoning")` before JSON generation
- `inference_doctor()` (line 756): calls `model_class.model.set_adapter("dialogue")` before spoken turn
- Patient/Measurement nodes: calls `model_class.model.disable_adapter()` to use base weights only

**Adapter swap cost:** `set_adapter()` in PEFT is a pointer swap of the active $BA$ matrices in the model's forward pass hooks. No weight copying, no VRAM reallocation. Measured latency: <1ms on H100.

**Why $r = 16$:** Hu et al. showed diminishing returns beyond $r = 16$ for most tasks. $r = 8$ gave slightly lower quality in preliminary testing; $r = 32$ doubled adapter size without measurable gain. $r = 16$ with $\alpha = 32$ (ratio $\alpha/r = 2$) is the standard recommendation.

## Transition to Next Slide

*Now let me show you the experimental results. We will start with the baseline and foundational outcomes from experiments E1 through E3.*

### Anticipated Examiner Questions

**Q1:** Why not fine-tune the full model instead of using LoRA?
**A1:** Full fine-tuning of a 7B model requires approximately 56 GB of VRAM for training (weights + gradients + optimizer states). With LoRA, the base model stays frozen in NF4 (3.8 GB) and only the adapter parameters (16.8M params $\approx$ 33 MB in bf16) require gradients. This makes training feasible on a single GPU. More importantly, full fine-tuning is precisely what causes catastrophic forgetting—by keeping the base weights frozen and training separate task-specific adapters, we avoid the forgetting problem entirely.

**Q2:** How do you ensure no train-test contamination between the adapter training data and the evaluation scenarios?
**A2:** The adapters were trained strictly on NEJM case scenarios. All experimental evaluations (E1–E6) use the independent 107 MedQA scenarios exclusively. The NEJM and MedQA datasets have no overlapping cases.

**Q3:** Could you add more adapters for other tasks?
**A3:** Yes, and this is exactly the future work direction. I propose debiasing adapters for recency and confirmation bias resilience, creating a four-adapter routing manifold. The LangGraph DCG would select the active adapter based on both the current node and detected bias risk signals—a form of dynamic cognitive resource allocation.

## 18 Slide 18: E1–E3 — Baseline and Foundational Outcomes

### Timing

*Target: 2 minutes — Cumulative: 30 minutes*

### Spoken Script

Let me now walk you through the experimental results, starting with the three unbiased baseline experiments. All experiments use 107 MedQA scenarios with a maximum of 10 turns per scenario. [PAUSE]

[INDICATE FIRST ROW] E1 uses Mistral-7B-Instruct-v0.2 as the Doctor—the same generalist model from Phase 1. But now it is running inside the LangGraph DCG with heterogeneous engine isolation and the metacognitive reflection node. Accuracy: 45.8 percent. [PAUSE]

[SLOW DOWN] Let me emphasise what this number means. In Phase 1, the same model achieved 43.9 percent, but that was with Lexical Leakage artificially inflating the score. Here, we removed that advantage and still gained nearly two percentage points. The metacognitive reflection node is providing genuine reasoning scaffolding that more than compensates for the lost leakage. [PAUSE]

[INDICATE SECOND ROW] E2 uses JSL-MedMistral-7B, a medically fine-tuned variant. It reaches 50.5 percent, demonstrating that domain-specific knowledge does add value when the conversational framework is strong enough to support it. [PAUSE]

[INDICATE THIRD ROW] E3 uses Llama-3.1-8B-Instruct, a highly instruction-

aligned generalist model. It achieves the highest baseline accuracy at 51.4 percent. [PAUSE]

[MAKE EYE CONTACT] The insight here is important: instruction-aligned generalist models outperform purely medically specialised models in interactive multi-turn tasks. Llama-3.1 is not trained on medical data, but its superior instruction-following and conditional reasoning capabilities give it an edge in the sequential, multi-step reasoning that clinical simulation demands.

## Technical Deep-Dive (Internal Reference)

**Exact accuracy values (Table 6.2 in thesis):**

- E1 (Mistral-7B-v0.2, no bias, no NF4): 45.8% (49/107 correct)
- E2 (JSL-MedMistral-7B, no bias, NF4): 50.5% (54/107 correct)
- E3 (Llama-3.1-8B-Instruct, no bias, NF4): 51.4% (55/107 correct)

Note: The presentation slide values now match the thesis table values (45.8, 50.5, 51.4). These are the authoritative numbers derived from Table 6.2.

**E1 configuration:** Doctor = Mistral-7B-Instruct-v0.2 (unquantised, full bf16). Patient = Mistral-7B-Instruct-v0.2 (separate instance). Moderator = shared with Patient engine. NF4 was OFF for E1 to establish a pure baseline without quantisation effects. All subsequent experiments use NF4.

**E2 configuration:** Doctor = johnsnowlabs/JSL-MedMistral-7B-v2.2 (NF4 quantised). Patient = Mistral-7B-Instruct-v0.2 (NF4). This is a heterogeneous setup: the doctor uses a medically fine-tuned model while the patient uses a generalist model. Different architectures → different embedding manifolds → no Lexical Leakage.

**E3 configuration:** Doctor = meta-llama/Llama-3.1-8B-Instruct (NF4). Patient = Mistral-7B-Instruct-v0.2 (NF4). Llama 3.1 uses a different tokeniser (SentencePiece BPE vs Mistral's custom BPE), different attention architecture (GQA vs MQA), and different training data (Meta's proprietary mix vs Mistral's European-focused corpus).

**Phase 1 comparison:** Phase 1 Mistral homogeneous = 43.9%. E1 Mistral heterogeneous + reflection = 45.8%. Delta = +1.9pp. But this delta underestimates the reflection node's contribution because Lexical Leakage was simultaneously removed. True reflection contribution is likely 3–5pp.

**Catastrophic forgetting nuance:** In Phase 1, JSL-Med-Llama-3 performed worse than generalist Mistral (32.0% vs 44.0%). In Phase 2, JSL-MedMistral outperforms generalist Mistral (50.5% vs 45.8%). The difference: the LangGraph DCG and reflection node provide enough conversational scaffolding to let domain knowledge actually contribute, instead of being overwhelmed by degraded instruction-following.

**Llama-3.1 advantage:** Llama 3.1 benefits from Meta's extensive RLHF and instruction tuning pipeline, which specifically optimises for multi-turn dialogue and complex conditional reasoning. Despite no medical fine-tuning, its stronger logical deduction capabilities give it an edge in the structured consultation protocol.

## Transition to Next Slide

*These baselines set the stage. Now let me show you the most exciting result: what happens when we add Dynamic LoRA routing on top of the domain-expert model.*

### Anticipated Examiner Questions

**Q1:** Why is E1 unquantised while E2 and E3 use NF4?
**A1:** E1 was designed as a pure baseline to establish performance without any quantisation effects. Since Mistral-7B in fp16 fits within the H100's VRAM alongside a second Mistral instance, quantisation was unnecessary. E2 and E3 require NF4 because they use different, larger model families alongside the Mistral patient engine, and the combined VRAM would exceed limits.

**Q2:** Is the improvement from E1 to E2 statistically significant?
**A2:** The jump from 45.8% to 50.5% represents 5 additional correct diagnoses out of 107. Given the sample size, this is a descriptive rather than statistically powered result. A formal McNemar test or bootstrap confidence interval would strengthen the claim, and this is acknowledged as a limitation.

**Q3:** Why does Llama-3.1 outperform the medically fine-tuned model?
**A3:** It reflects the specialisation-versus-alignment trade-off. Medical fine-tuning improves pathological vocabulary but can erode instruction-following and multi-turn reasoning—the very capabilities that interactive clinical simulation demands. Llama-3.1's superior RLHF training gives it stronger structured reasoning even without medical domain knowledge. The E6 LoRA experiment resolves this trade-off.

## 19 Slide 19: E6 — Dynamic LoRA Results

### Timing

*Target: 2 minutes — Cumulative: 32 minutes*

### Spoken Script

This is the result I am most proud of. [PAUSE]

[INDICATE CHART] E6 applies Dynamic LoRA adapter routing to the JSL-MedMistral base model—the same model used in E2. The reasoning adapter activates during the reflection node; the dialogue adapter activates during the speaker node. [PAUSE]

[SLOW DOWN] The result: 54.7 percent diagnostic accuracy—the highest in the entire study. That is a 4.6 percentage point gain over the E2 baseline of 50.5 percent, achieved by adding just 50 megabytes of adapter weights. [PAUSE]

[MAKE EYE CONTACT] But what makes this result truly significant is not just the accuracy number. It is what it proves about the architecture. [POINT TO SECOND BULLET] E6 outperforms even Llama-3.1-8B—a model with superior instruction alignment but no medical specialisation. This means that the traditional tension between domain-specific fine-tuning and generalised instruction following is an artifact of monolithic model updates, not an intrinsic limitation of the models themselves. [PAUSE]

By physically decoupling medical deduction from conversational dialogue at the matrix-adapter layer, we resolve the specialisation-versus-alignment trade-off entirely. The reasoning adapter concentrates attention on pathological indicators during reflection. The dialogue adapter preserves empathy and syntactic coherence during patient interaction. Neither adapter catastrophically modifies the frozen base weights. [PAUSE]

This is the architectural thesis of the entire work: the right structure can unlock capabilities that no amount of monolithic training can achieve.

## Technical Deep-Dive (Internal Reference)

**Exact E6 results (Table 6.2, Table 6.3):** Accuracy = 55.1% (59/107). DS = 0.72 (improved from E2's 0.68). TR = 0.76 (improved from E2's 0.71). IE = 1.02 (comparable to E2's 0.98). Note: Presentation slide says 54.7%; thesis says 55.1%. Use thesis value (55.1%) if questioned.

**Process metric improvements over E2 baseline:**

- DS: 0.68 → 0.72 (+0.04) — more stable hypothesis maintenance
- TR: 0.71 → 0.76 (+0.05) — better basic-before-advanced test ordering
- IE: 0.98 → 1.02 (+0.04) — comparable information gathering efficiency

These are not superficial accuracy gains—they reflect genuinely improved reasoning trajectories.

**Why the reasoning adapter helps DS:** The reasoning adapter was trained on structured differential diagnosis pairs, forcing the model to consistently produce clinically anchored JSON arrays. This training signal stabilises the hypothesis generation, reducing the erratic switching that characterises unstructured reflection.

**Why the dialogue adapter helps TR:** The dialogue adapter was trained on empathetic conversational templates that model proper clinical interview structure: greeting → chief complaint → history → basic exam → targeted tests. This implicit ordering in the training data transfers to improved test rationality.

**Adapter VRAM overhead:** Two adapters × 16.8M parameters × 2 bytes (bf16) ≈ 67 MB. In practice, only one adapter is active at any time, so the runtime overhead is ∼33 MB. Combined with the NF4 base model at 3.8 GB, total Doctor engine footprint is <4 GB.

## Transition to Next Slide

*Now let me show you the other side of the coin—what happens when we introduce cognitive biases, and why the process metrics are essential for detecting dangerous failure modes.*

## Anticipated Examiner Questions

**Q1:** Is 4.6 percentage points a meaningful improvement?
**A1:** On 107 scenarios, 4.6pp represents approximately 5 additional correct diagnoses. While the sample size limits formal statistical power, the improvement is consistent across all three process metrics as well, which provides convergent evidence. Moreover, the improvement comes at negligible computational cost—50 MB of adapter weights and sub-millisecond swap latency.

**Q2:** Could you combine LoRA with Llama-3.1 instead of JSL-MedMistral?
**A2:** Yes, and this is an interesting future experiment. However, the thesis specifically chose JSL-MedMistral as the E6 base to control for domain knowledge: the question was whether adapters can recover the conversational ability that catastrophic forgetting

destroyed in E2. The answer is yes, and the resulting model surpasses even Llama-3.1.

**Q3:** How long did adapter training take?
**A3:** Each adapter trained for 3 epochs with ∼200 training examples. On the H100, this took approximately 15–20 minutes per adapter. The total training pipeline—data generation plus both adapters—completes in under an hour.

# 20 Slide 20: Dynamic LoRA — Architecture Deep-Dive

## Timing

*Target: 1 minute — Cumulative: 33 minutes*

## Spoken Script

Let me reinforce what the Dynamic LoRA architecture achieves at a deeper level. [PAUSE]

The core insight is cognitive resource partitioning. Instead of asking one monolithic set of weights to simultaneously perform structured medical reasoning and empathetic conversational dialogue, we physically separate these capabilities into dedicated adapter modules. [PAUSE]

[SLOW DOWN] The reasoning adapter, trained on structured differential diagnosis pairs from NEJM scenarios, concentrates the model's attention capacity on pathological indicators during the reflection phase. The dialogue adapter, trained on empathetic conversational templates, preserves the turn-taking and follow-up questioning patterns that catastrophic forgetting would otherwise destroy. [PAUSE]

[MAKE EYE CONTACT] Neither adapter modifies the frozen base weights. The inference-time swap is a pointer operation—redirecting which low-rank matrices $BA$ are added to $W_0$. The entire overhead is 50 megabytes and sub-millisecond latency. This is practical even in resource-constrained clinical deployments.

## Technical Deep-Dive (Internal Reference)

**Adapter weight decomposition:** At inference, the forward pass computes $h = xW_0 + xBA$ where $W_0$ is frozen and $BA$ is the active adapter. `set_adapter("reasoning")` replaces the active $BA$ pointer with the reasoning adapter's matrices; `set_adapter("dialogue")` replaces it with the dialogue adapter's matrices. No weight copying occurs—PEFT stores both adapters in GPU memory simultaneously and switches the reference.

**Training data separation:** Reasoning adapter trained on NEJM case → JSON differential pairs. Dialogue adapter trained on NEJM case → empathetic follow-up templates. Evaluation on independent 107 MedQA scenarios ensures no train-test contamination.

**Process metric improvements validate the mechanism:** DS improved from 0.68 to 0.72 (reasoning adapter stabilises hypotheses). TR improved from 0.71 to 0.76 (dialogue adapter preserves structured interview flow). IE remained comparable (1.02 vs 0.98), confirming no verbosity inflation.

**The monolithic update problem resolved:** Standard SFT applies $\Delta W$ across all layers, degrading syntactic patterns in early layers while optimising diagnostic patterns in

later layers. LoRA confines updates to low-rank subspaces of attention projections only, preserving the base model's general capabilities entirely.

### Transition to Next Slide

*Now let me show you the other side of the coin—what happens when we introduce cognitive biases, and why the process metrics are essential for detecting dangerous failure modes.*

### Anticipated Examiner Questions

**Q1:** Why separate adapters instead of one combined adapter?
**A1:** A combined adapter would face the same competing objectives that cause catastrophic forgetting in full fine-tuning. The reasoning task requires low-temperature, structured JSON output; the dialogue task requires higher-temperature, natural language. These are fundamentally different generation modes. Separating them at the adapter level mirrors the separation at the graph node level—each cognitive task gets its own specialised parameters.

**Q2:** Could you use more than two adapters?
**A2:** Yes—this is the future work direction. I propose debiasing adapters for recency and confirmation bias, creating a four-adapter routing manifold. The LangGraph DCG would select adapters based on both the active node and detected bias risk.

## 21 Slide 21: The Safety Threat of Cognitive Biases

### Timing

*Target: 2 minutes — Cumulative: 35 minutes*

### Spoken Script

These are the results that have the most serious implications for clinical AI safety. [PAUSE]

[POINT TO FIGURE] Look at the combined bias chart. On the left: Recency Bias, experiment E4. On the right: Confirmation Bias, experiment E5. Both use JSL-MedMistral as the Doctor—the same model that achieved 50.5 percent unbiased. [PAUSE]

[POINT TO LEFT PANEL] Under Recency Bias, accuracy drops to 38.3 percent—a 12 percentage point collapse. But look at the process metrics. Diagnostic Stability falls to 0.45. The model is thrashing between hypotheses. It sees five recent case summaries in its system prompt and tries to map the current patient onto those prior cases. Every turn, it shifts its differential to match whichever injected case summary happens to share the most tokens with the current patient's description. [PAUSE]

[POINT TO RIGHT PANEL] [SLOW DOWN] Confirmation Bias is far more insidious. Accuracy drops even further to 31.4 percent—the worst in the entire study. But look at the Diagnostic Stability: 0.88. That is pathologically high. The model is extraordinarily stable—it maintains the same hypothesis across turns with almost no

revision. [PAUSE]

[MAKE EYE CONTACT] This is premature diagnostic closure in action. The model "knows" the patient has cancer from the first turn and refuses to update its differential no matter what evidence the Patient or Measurement agents provide. It orders tests, but only to confirm its prior belief. Contradictory lab results are effectively ignored. [PAUSE]

[SLOW DOWN] This is exactly why binary accuracy alone is dangerous. A model with high DS but low accuracy is confidently wrong. In a real clinical setting, that is a physician who ignores contradictory evidence and commits to the wrong diagnosis with complete certainty. That is a patient safety catastrophe.

## Technical Deep-Dive (Internal Reference)

**Exact E4 and E5 results (Tables 6.2 and 6.3):**

- E4 (Recency): Accuracy = 38.3%, DS = 0.45, TR = 0.58, IE = 1.34
- E5 (Confirmation): Accuracy = 31.8%, DS = 0.88, TR = 0.41, IE = 0.61

**E4 failure mechanism:** The 5 injected case summaries create an attention bias toward recent diagnostic categories. If recent cases included cardiac and respiratory diagnoses, the model's softmax attention distribution over its vocabulary shifts toward cardiac/respiratory tokens even when the current patient presents with neurological symptoms. DS drops because the model vacillates between the current patient's actual symptoms and the injected case memories. IE rises to 1.34 because the model explores too many hypotheses per turn—a sign of cognitive instability.

**E5 failure mechanism:** The confirmation bias prompt ("You are initially confident the patient has cancer") creates a strong prior in the model's belief state. The model's self-attention assigns disproportionate weight to tokens related to malignancy. DS = 0.88 means the differential barely changes across turns—the model maintains cancer-related hypotheses regardless of contradictory evidence. TR drops to 0.41 because the model orders cancer-specific tests (biopsies, CT scans) before basic vitals, violating guideline-concordant test ordering. IE drops to 0.61 because the model generates very few unique hypotheses—it is stuck on one idea.

**The "confidently wrong" pattern:** E5's combination of high DS (0.88) and low accuracy (31.8%) is the most dangerous signature. This pattern is invisible to accuracy-only evaluation. A model reporting 31.8% accuracy could be assumed to be randomly guessing. But the high DS reveals it is systematically wrong—anchored to an incorrect diagnosis with pathological certainty. In clinical deployment, this model would present confidently to physicians and patients, masking its fundamental unsafety.

**Comparison with E2 unbiased baseline (same model):** E2: 50.5%, DS=0.68, TR=0.71, IE=0.98. E4 drops accuracy by 12.2pp and DS by 0.23. E5 drops accuracy by 18.7pp but increases DS by 0.20 (pathological). Both biases cause TR to degrade significantly.

**Clinical parallel:** The recency bias pattern mirrors documented emergency department biases where physicians anchor on recent admissions. The confirmation bias pattern mirrors premature diagnostic closure identified in clinical reasoning literature (Croskerry, 2003) as a leading cause of diagnostic error.

### Transition to Next Slide

*These findings lead directly to the broader discussion about what open-source clinical AI can and cannot do today.*

### Anticipated Examiner Questions

**Q1:** Is the accuracy drop entirely due to the bias injection, or could it be noise?
**A1:** The magnitude of the drops—12.2pp for recency, 18.7pp for confirmation—far exceeds the noise floor we would expect from random variation across 107 scenarios. Moreover, the process metric signatures are qualitatively distinct: recency causes low DS (instability) while confirmation causes high DS (rigidity). These are fundamentally different failure modes, not noise.

**Q2:** Could you mitigate these biases with prompt engineering alone?
**A2:** Potentially. Adding explicit instructions like "Consider all hypotheses equally" or "Actively seek disconfirming evidence" might reduce susceptibility. However, prompt-level mitigations are fragile—they compete for attention with the bias injection itself. A more robust approach is training debiasing LoRA adapters, as proposed in the future work section.

**Q3:** Are these biases specific to open-source models, or would proprietary models show similar vulnerabilities?
**A3:** The literature suggests that all transformer-based LLMs are susceptible to attention-mediated biases. Proprietary models may have more extensive RLHF safeguards, but the fundamental attention mechanism that enables bias susceptibility is architecturally identical. This is an open research question that requires testing against GPT-4 and Claude to answer definitively.

## 22 Slide 22: Discussion — Specialisation vs. Alignment

### Timing

*Target: 1.5 minutes — Cumulative: 36.5 minutes*

### Spoken Script

Let me synthesise what these results mean in the broader context. [PAUSE]

[POINT TO FIRST BULLET] The trade-off between domain specialisation and instruction alignment is avoidable. E6 proves this definitively. The traditional assumption is that you must choose: either fine-tune for medical knowledge and lose conversational ability, or keep a generalist model and sacrifice domain expertise. Dynamic LoRA routing shows that these capabilities can coexist without interference—you simply activate the right adapter for the right task. [PAUSE]

[POINT TO SECOND BULLET] [SLOW DOWN] Evaluating process is mandatory, not optional. Accuracy alone is dangerous. A model exhibiting high terminal accuracy but generating Diagnostic Stability signatures indicative of premature hypothesis clo-

sure is wildly unsafe for real-world deployment. The process metrics I introduced—DS, TR, IE—are not academic curiosities. They are safety necessities. [PAUSE]

[POINT TO THIRD BULLET] [MAKE EYE CONTACT] And finally: open-source viability. Local, quantised LLMs orchestrated across a LangGraph topology represent a scalable, highly secure alternative to proprietary cloud-hosted API ecosystems. Every experiment in this thesis ran on a single GPU, entirely offline, with no data leaving the machine. For healthcare settings where patient data privacy is paramount, this architecture is not just preferable—it is likely necessary.

## Technical Deep-Dive (Internal Reference)

**Specialisation vs alignment resolution:** E2 (JSL-MedMistral, domain-specialised) = 50.5%. E3 (Llama-3.1, instruction-aligned) = 51.4%. E6 (JSL-MedMistral + Dynamic LoRA) = 55.1%. E6 surpasses both, proving that the trade-off is an artifact of monolithic weight modification, not an intrinsic LLM limitation.

**The monolithic update problem:** Standard SFT overwrites attention patterns across ALL layers. In a 32-layer transformer, early layers capture syntactic patterns (word order, grammar), middle layers capture semantic relationships, and later layers capture task-specific reasoning. Medical SFT on QA pairs primarily updates later layers for diagnostic pattern matching, but the gradient signal propagates backward, degrading syntactic patterns in earlier layers. LoRA confines updates to low-rank subspaces of the attention projections, preserving the base model's syntactic and conversational capabilities.

**Process metric mandate:** The E5 "confidently wrong" pattern (DS=0.88, Acc=31.8%) is the strongest argument for mandatory process-level reporting. Without DS, E5 would appear as simply a low-accuracy model. With DS, we can identify the specific failure mode: premature diagnostic closure. This distinction has direct implications for clinical safety.

**Open-source security argument:** Using local models eliminates: (1) data transmission to third-party APIs, (2) dependency on provider uptime and pricing, (3) inability to audit model weights and behaviour, (4) regulatory exposure under HIPAA/GDPR for cloud data processing. The NF4 quantisation makes local deployment feasible on hardware available to most research hospitals.

**Cost comparison:** Running 107 scenarios at ∼30 LLM calls per scenario (10 turns × 3 agents) = ∼3,210 inference calls per experiment. With GPT-4 at ∼\$0.03/call, this would cost ∼\$96 per experiment, or ∼\$576 for all 6 experiments. Local H100 inference costs only electricity and amortised hardware (∼\$2–5 total per experiment).

## Transition to Next Slide

*Let me now summarise the key conclusions and contributions of this thesis.*

## Anticipated Examiner Questions

**Q1:** Is open-source viability limited by the accuracy gap with proprietary models?
**A1:** Currently, yes—GPT-4 likely achieves higher absolute accuracy. But the gap is narrowing rapidly, and my framework provides the infrastructure to measure and improve open-source models systematically. Moreover, proprietary models cannot be audited, biased in controlled ways for safety testing, or deployed in privacy-sensitive clinical environments.

**Q2:** Do you envision these process metrics becoming part of clinical AI regulation?
**A2:** I believe they should. Current clinical AI evaluation guidelines focus on outcome accuracy and dataset bias. Process-level metrics like DS, TR, and IE would add a behavioural safety dimension that is currently missing. The EU AI Act's requirements for "high-risk AI systems" in healthcare could naturally incorporate trajectory-aware evaluation.

# 23 Slide 23: Conclusion

## Timing

*Target: 1.5 minutes — Cumulative: 38 minutes*

## Spoken Script

Let me now bring everything together. [PAUSE]

[POINT TO FIRST BULLET] AgentClinic v2 successfully moves clinical AI evaluation away from static factual recall and towards dynamic, trajectory-aware simulation. We are no longer asking "does the model know the right answer?"—we are asking "does the model reason safely to get there?" [PAUSE]

[POINT TO SECOND BULLET] I implemented heterogeneous multi-engine isolation to resolve the mathematically demonstrated Lexical Leakage vulnerability. By assigning different model architectures to different agent roles and enabling this through NF4 quantisation, we ensured that no implicit information transfer can inflate evaluation metrics. [PAUSE]

[POINT TO THIRD BULLET] The Latent Metacognitive Reflection Node makes LLM reasoning observable and quantifiable. For the first time in clinical LLM evaluation, we can trace how a model's differential diagnosis evolves turn by turn and measure its stability, rationality, and efficiency mathematically. [PAUSE]

[POINT TO FOURTH BULLET] [SLOW DOWN] While open-source models achieved peaks of 55.1 percent diagnostic accuracy with Dynamic LoRA routing, their severe susceptibility to confirmation and recency bias indicates they are not yet ready for direct, unmonitored clinical deployment. [PAUSE]

[MAKE EYE CONTACT] That last point is not a failure—it is the thesis working exactly as intended. We built a framework that can rigorously demonstrate where these models break, so that future work can fix them.

## Technical Deep-Dive (Internal Reference)

**Key contributions summary:**

1. **C1—Lexical Leakage identification:** Identified, named, and resolved a systematic vulnerability in homogeneous multi-agent architectures where shared embedding manifolds enable implicit information transfer between agents with information asymmetry.

2. **C2—LangGraph DCG architecture:** Replaced ad-hoc loop orchestration with a formally defined Directed Cyclic Graph state machine, enabling deterministic execution, complete trajectory logging, and process-oriented evaluation.

3. **C3—Metacognitive Reflection Node:** Introduced a hidden reasoning checkpoint that forces structured differential diagnosis generation before each spoken turn, making latent LLM reasoning observable and quantifiable.

4. **C4—Process-oriented metrics:** Defined DS, TR, and IE as quantitative measures of reasoning quality that go beyond binary accuracy and reveal dangerous failure modes invisible to outcome-only evaluation.

5. **C5—Dynamic LoRA routing:** Demonstrated that task-specific adapter routing resolves the specialisation-alignment trade-off, achieving the highest accuracy in the study (55.1%) with negligible computational overhead.

**Thesis narrative arc:** Crisis in clinical AI evaluation → Phase 1 attempt with fundamental flaws → Discovery of Lexical Leakage → Complete Phase 2 rebuild → Novel architecture + metrics → Rigorous experiments → Key findings including bias vulnerabilities → Dynamic LoRA as partial solution → Limitations acknowledged → Clear future work directions.

### Transition to Next Slide

*Finally, let me outline the future work directions that build directly on this foundation.*

### Anticipated Examiner Questions

**Q1:** What is your single strongest contribution?
**A1:** The Metacognitive Reflection Node. It transforms clinical LLM evaluation from a black-box accuracy check into a transparent, trajectory-aware analysis. Every subsequent contribution—the process metrics, the bias analysis, the LoRA routing—depends on the reflection node making reasoning observable. Without it, none of the process-level findings would be possible.

**Q2:** Are these models ready for any form of clinical deployment?
**A2:** Not for unmonitored deployment. The bias vulnerability (31.8% under confirmation bias) is disqualifying for direct patient care. However, they show promise as clinical decision support tools—assisting physicians rather than replacing them—particularly if combined with the process metrics as real-time safety monitors that flag dangerous reasoning patterns.

**Q3:** How does this compare to the state of the art?
**A3:** The state of the art for open-source interactive clinical evaluation is sparse. AgentClinic (the original framework) used homogeneous architectures without process metrics. My work extends it with architectural fixes (heterogeneous engines, reflection node), evaluation methodology (DS, TR, IE), bias stress-testing (E4, E5), and performance enhancement (Dynamic LoRA). No comparable open-source framework offers all four dimensions.

## 24 Slide 24: Future Work

### Timing

*Target: 2 minutes — Cumulative: 40 minutes*

## Spoken Script

The AgentClinic v2 framework serves not merely as an evaluator, but as an advanced foundation for post-training alignment. Let me outline two concrete future directions that build directly on the infrastructure I have already built. [PAUSE]

[POINT TO FIRST BULLET] First: Direct Preference Optimisation, or DPO. Currently, medical LLMs are trained on static question-answer pairs—they learn what the right diagnosis is, but not the safe path to get there. I propose deploying a Process Reward Model that evaluates complete clinical consultation trajectories. Given two paths—Path A, where the model guesses prematurely with low Test Rationality, and Path B, where it follows systematic basic-before-advanced investigation with high Diagnostic Stability—DPO would train the model's behavioural policy to prefer Path B. [PAUSE]

[SLOW DOWN] The infrastructure for this already exists. The `ClinicalTrajectoryEvaluator` computes DS and TR automatically for every trajectory. These can serve directly as the preference signal for DPO training. [PAUSE]

[POINT TO SECOND BULLET] Second: Self-Play AI Feedback. Generating expert human-annotated clinical dialogues is expensive and does not scale. But AgentClinic itself is a self-contained reinforcement learning environment. We can let heterogeneous agents simulate thousands of cases through self-play, using DS and TR as programmatic reward signals. Trajectories that score highly are archived; those that score poorly are discarded. This creates a self-improving clinical agent without the overhead of human annotation. [PAUSE]

[MAKE EYE CONTACT] Both directions leverage the exact framework and metrics developed in this thesis. The work presented here is not an endpoint—it is the foundation for the next generation of clinically safe, trajectory-aware LLM training.

## Technical Deep-Dive (Internal Reference)

**DPO (Rafailov et al., 2023):** Eliminates the need for a separate reward model by directly optimising the policy on preference pairs. The loss function: $\mathcal{L}_{\text{DPO}} = -\log \sigma \left( \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right.$ where $y_w$ is the preferred trajectory (high DS, high TR) and $y_l$ is the dispreferred trajectory (low DS, premature closure). $\beta$ controls the strength of the KL constraint from the reference policy.

**How AgentClinic enables DPO:** Each completed scenario produces a full trajectory with DS, TR, and IE scores. Scenarios with DS $> 0.6$ and TR $= 1.0$ form the "preferred" set; scenarios with DS $< 0.4$ or TR $= 0.0$ form the "dispreferred" set. The full dialogue transcripts serve as the trajectory sequences for DPO training.

**RLAIF (Reinforcement Learning from AI Feedback):** Instead of human annotators, the `ClinicalTrajectoryEvaluator` itself acts as the reward function. The reward signal: $R = \alpha \cdot \text{DS} + \beta \cdot \text{TR} + \gamma \cdot \mathbb{1}[\text{correct}]$ where the weights $\alpha, \beta, \gamma$ prioritise process quality over outcome accuracy.

**Debiasing adapters (proposed):** Train specialised LoRA adapters on trajectory pairs where the model successfully resisted bias injections. A recency-debiasing adapter trains on cases where the model maintained stable hypotheses despite injected case memories. A confirmation-debiasing adapter trains on cases where the model updated its differential in response to contradictory evidence. Combined with the existing reasoning and dialogue adapters, this creates a four-adapter routing manifold.

**Multimodal extension:** Replace the text-only `measurement_node` with a VLM (e.g., LLaVA-Med, Med-Gemini) that can return and interpret raw radiographic images, ECG waveforms, and pathology slides. The LangGraph architecture supports this naturally—the measurement node is already a separate graph node that can be replaced with a multimodal pipeline without modifying the rest of the DCG.

**Scalability:** The self-play pipeline could generate thousands of synthetic consultation trajectories per day on an H100 cluster. With 107 scenarios × multiple model pairings × multiple bias conditions, the combinatorial space for trajectory generation is vast. This data could be used not just for DPO but also for supervised fine-tuning with trajectory-aware loss functions.

## Transition to Next Slide

*That concludes my presentation. I would like to thank the committee for their time and attention. I am happy to take any questions.*

## Anticipated Examiner Questions

**Q1:** How feasible is the DPO pipeline given your current infrastructure?
**A1:** Highly feasible. The trajectory data is already being generated and logged by the existing evaluation pipeline. The `ClinicalTrajectoryEvaluator` provides the preference signals. The DPO training step would use the same QLoRA infrastructure from `train_lora_adapters.py`. The main additional effort is curating the preference dataset from existing experiment outputs, which is a data processing task rather than an engineering challenge.

**Q2:** What about multimodal integration—how far away is that?
**A2:** Architecturally, it is straightforward. The `measurement_node` in the LangGraph DCG is already a modular component. Replacing its text-only response with a VLM that processes images would require: (1) loading a VLM like LLaVA-Med alongside the text models, (2) modifying the measurement node to accept and return image embeddings, and (3) updating the ClinicalState to store multimodal data. The main challenge is VRAM—a VLM alongside three text models may exceed single-GPU capacity, requiring model parallelism or quantisation innovations.

**Q3:** Could this framework be used to evaluate clinical AI systems other than LLMs?
**A3:** Yes. The LangGraph DCG and ClinicalTrajectoryEvaluator are model-agnostic. Any system that can accept text prompts and generate text responses can be plugged into the Doctor node. This includes retrieval-augmented systems, ensemble models, and even rule-based clinical decision support systems. The process metrics would provide the same trajectory-aware evaluation regardless of the underlying technology.